



杭州电子科技大学
HANGZHOU DIANZI UNIVERSITY

StarryX

Operating System Design Manual

陆晨涛、祖启松

July 31, 2025

目录

1. 概述	4
1.1. 系统简介	4
1.2. 系统架构	5
1.3. 系统完成情况	6
2. 启动与初始化	7
2.1. 硬件初始化	7
2.2. 组件初始化	8
2.3. 宏内核初始化	9
3. 进程管理	10
3.1. 整体架构	10
3.2. 任务结构	11
3.3. 任务调度	12
3.4. 任务扩展	13
3.5. 进程通信	15
4. 内存管理	18
4.1. 整体架构	18
4.2. 内存分配器	18
4.3. 地址空间管理	20
4.4. 延迟分配技术	22
4.5. 用户地址访问	24
4.6. 页面异常处理	25
5. 文件系统	26
5.1. 整体架构	26
5.2. 虚拟文件系统	27
5.3. 文件系统实例	29
5.4. 缓存设计	31
5.5. 抽象文件	33

5.6. 伪文件系统	35
6. 网络	38
6.1. 整体架构	38
6.2. TCP 套接字	40
6.3. UDP 套接字	41
6.4. 系统调用实现	41
7. 组件化与抽象解耦	42
7.1. 设计理念	42
7.2. XUSPACE	42
7.3. XPROCESS	44
7.4. XCACHE	47
7.5. XVMA	49
7.6. XSIGNAL	50

1. 概述

1.1. 系统简介

StarryX 操作系统是基于组件化操作系统 ArceOS 的宏内核扩展实现，完整实现了进程管理，内存管理，文件系统，信号系统等模块，通过硬件抽象层 axhal 能够运行在四个架构上（riscv64 / loongarch64 / x86_64 / aarch64），并成功移植到 riscv visionfive 和 loongarch 2K1000 硬件平台。

ArceOS 采用模块化设计，将操作系统功能拆分为可重用的组件，允许开发者根据特定场景需求灵活组合功能模块。StarryX 的设计理念是将 ArceOS 单内核(Unikernel)的组件化优势与宏内核的高性能特性相结合，通过在 ArceOS 的单内核架构上扩展实现宏内核的任务管理、内存管理等核心功能，构建一个高效、灵活且支持 Linux 应用兼容的组件化宏内核操作系统。StarryX 的设计理念基于以下核心原则：

- 1. 强调组件化与模块化，通过将任务管理、内存管理等功能封装为独立组件，降低模块间耦合度，提升系统的可维护性和可扩展性；
- 2. 注重性能优化，保留宏内核在系统调用和资源管理方面的高效性，同时通过 ArceOS 的组件化框架减少不必要的抽象开销；
- 3. 追求场景适配性，支持用户根据物联网、云计算或嵌入式设备等不同场景需求，灵活定制内核功能。通过以 ArceOS 为基座，StarryX 旨在为开发者提供一个高效、安全、可定制的宏内核操作系统平台，满足多样化场景下对性能、灵活性和兼容性的需求。

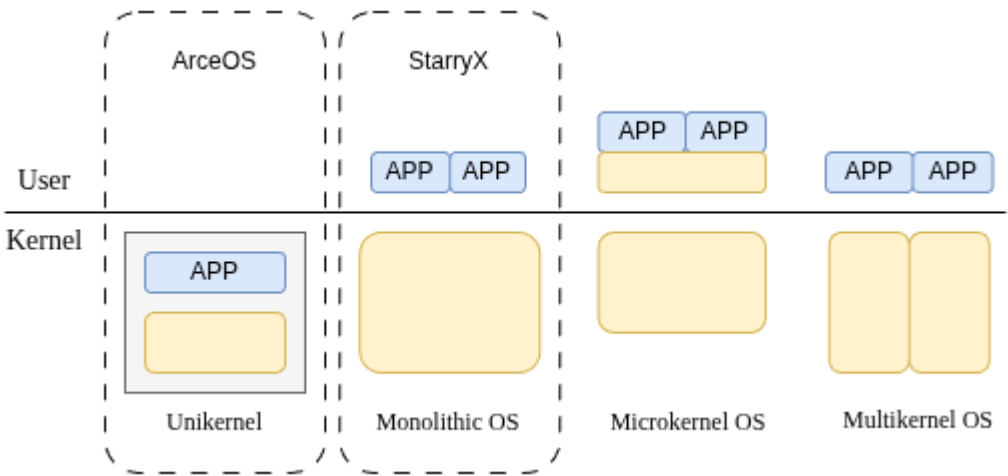


图 1.1 Monolithic 架构图

1.2. 系统架构

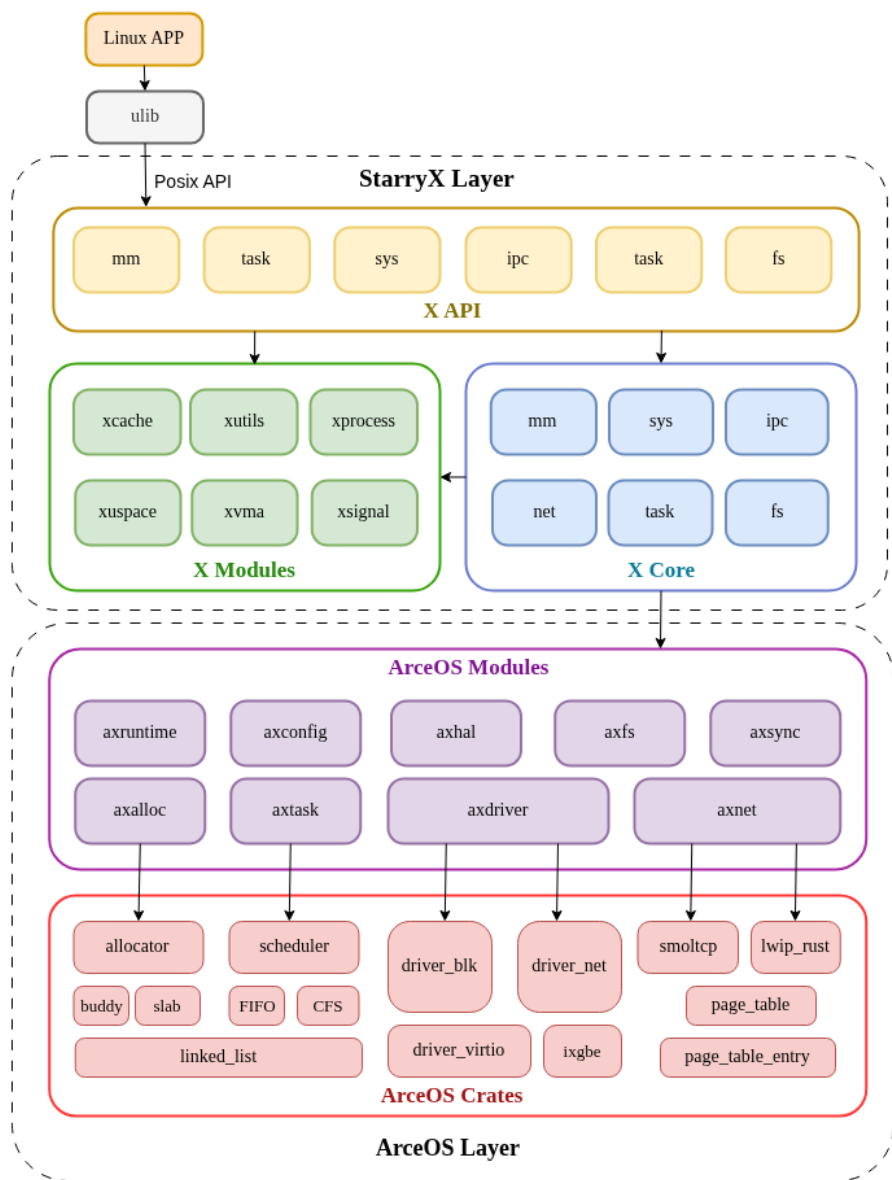


图 1.2 StarryX 架构图

StarryX 的系统架构主要分为两层：

1. 底层为 ArceOS Layer，ArceOS 的核心功能主要由各个模块构成，包括 axruntime、axconfig、axalloc、axfs、axsync、axtask、axdriver、axnet 等，这些模块由与具体操作系统无关的基础组件构成；ArceOS layer 为 StarryX 提供了内核的基础功能；

2. 上层为 StarryX Layer，StarryX 在 ArceOS 提供的内核基础服务上扩展宏内核相关功能，其主要由三个模块 X Core、X Modules 和 X API 构成，其中 X Modules 是实现了宏内核功能的可供复用的基础组件、X Core 实现了宏内核基础功能、X API 通过 X Core 和 X Modules 提供的服务实现了标准 POSIX 接口。

对于 StarryX layer，X Core 是实现宏内核功能的核心，主要实现了宏内核的进程管理、文件系统、内存管理、系统管理、进程通信和网络模块。

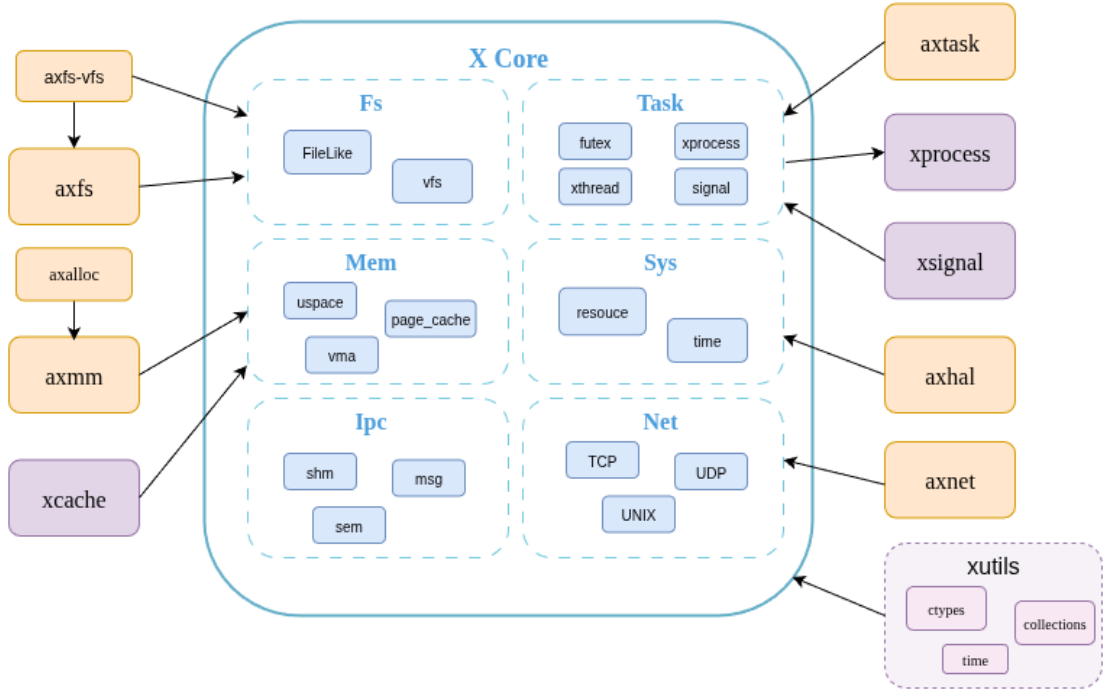


图 1.3 Xcore 架构图

1.3. 系统完成情况

截至目前，StarryX 共实现约 200 项系统调用，包括进程管理、文件系统、内存管理、网络等各个模块的系统调用，能够运行官方测试中的大量 LTP 测例以及 LTP 外的所有测例，并能够运行 Redis、Git、Bash 等 Linux 应用。

StarryX 的各个子模块实现均较为完善，完成情况如下：

子模块	完成情况
文件系统	类 Linux 的 VFS 设计
	丰富的抽象文件支持和完善的伪文件系统实现
	支持 EXT4 与 FAT 文件系统
	模块解耦的页缓存设计
内存管理	懒分配、写时复制
	模块解耦的用户空间访问
	模块解耦的文件映射管理
进程管理	多核场景下的负载均衡
	多种调度方式支持

	完整实现的 System V 进程通信
	模块解耦的进程管理
网络模块	支持 TCP 和 UDP 套接字
	实现端口复用
信号系统	模块解耦的信号系统
	可被信号中断的系统调用

2. 启动与初始化

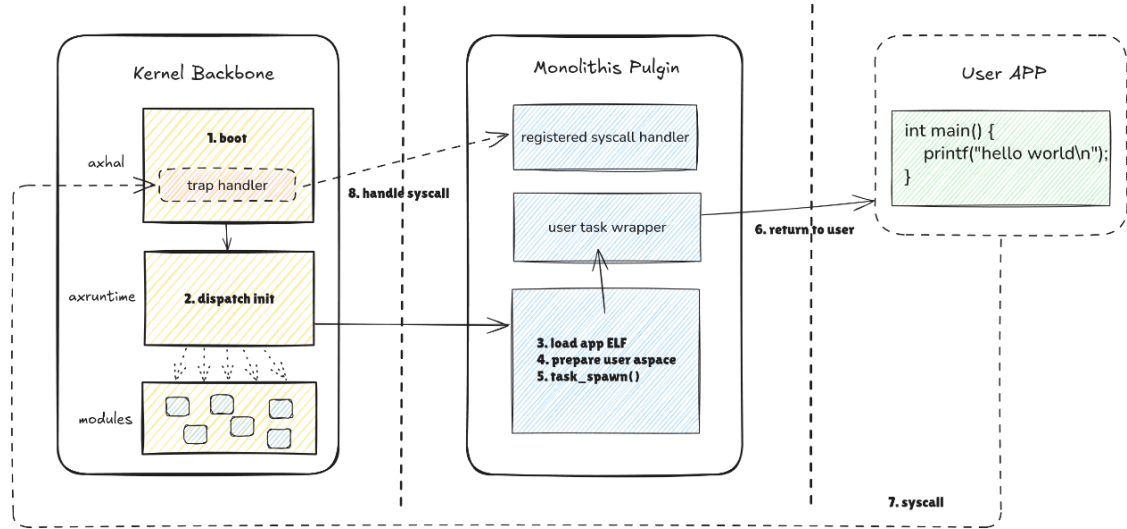


图 2.1 StarryX 启动流程图

StarryX 的启动流程涉及从 ArceOS Layer 到 StarryX Layer 再到用户程序，本部分将详细解释 StarryX 的启动与初始化过程，层次清晰地展现 StarryX 如何利用 ArceOS 的内核基础功能、扩展宏内核功能、服务用户程序。

2.1. 硬件初始化

`axhal` 模块抽象了底层硬件与平台，StarryX 从链接脚本的特定位置启动，转到特定 arch 的 `_start` 函数完成 CPU 与 mmu 初始化后跳转到硬件平台的 `rust_entry`，完成对硬件中断、时钟、串口等的硬件初始化后跳转到 `axruntime` 模块进行组件初始化：

```
1. // riscv qemu
2. extern "C" fn _start() -> ! {
3.     1. save hartid
4.     2. save DTB pointer
5.     3. setup boot stack
6.     4. setup boot page table and enabel MMU
```

```

7.     5. call rust_entry(hartid, dtb)
8. }
9.
10. extern "C" fn rust_entry(cpu_id: usize, dtb: usize) {
11.     1. clear .bss
12.     2. init CPU set stdev
13.     3. init timer / console
14.     4. call rust_main(hartid, dtb)
15. }

```

2.2. 组件初始化

axruntime 模块确保内核运行环境就绪并为宏内核扩展提供支撑，完成了 kernel backbone 的初始化过程。其启动流程包括以下步骤：

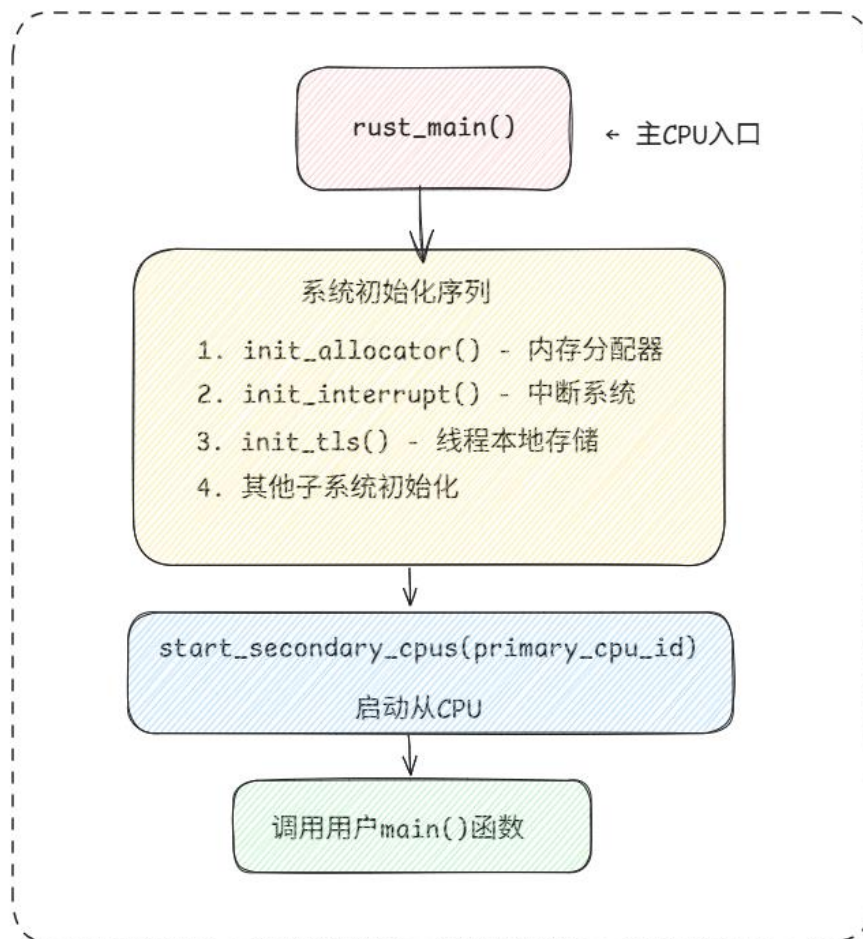


图 2.2 组件初始化过程

```

1. pub extern "C" fn rust_main(cpu_id: usize, dtb: usize) {
2.     axlog::init(); // 日志系统初始化
3.     init_allocator(); // 内存分配器初始化

```



```

4.    axmm::init();                // 虚拟内存管理初始化
5.    axtask::init_scheduler();    // 多任务调度器初始化
6.    axdriver::init();           // 驱动初始化
7.    asfs::new_root();           // 文件系统初始化
8.    asnet::init();              // 网络初始化
9.    mp::start();                // 多核初始化
10.   main()
11. }

```

完成 kernel backbone 的初始化流程后将进入内核扩展的初始化阶段，对于单内核来说，用户 APP 将接管 main 函数的运行，对于宏内核来说，StarryX 的 main 函数将接管 main 函数的运行，并完成宏内核的初始化。

2.3. 宏内核初始化

StarryX 的 main 函数接管内核运行后将各个子模块完成初始化，之后开始运行用户程序

```

1. fn main() {
2.     xprocess::new_init();        // 创建初始化进程
3.     xcore::fs::vfs::init_root(); // 初始化伪文件系统
4.     xcore::fs::fd::init_stdio(); // 初始化 FD 表
5.     run_user_app()               // 运行用户程序
6. }

```

对于运行的单个程序，StarryX 通过 run_user_task 构造可运行的 task，并调度运行，其主要经历以下几个过程：

1. 构建用户空间，映射内核区域
2. 加载并映射用户程序的 `ELF` 文件，返回入口地址。
3. 构造上下文并模拟返回
4. 初始化命名空间资源
5. 创建新进程
6. 为新进程创建线程
7. 等待进程运行结束并返回退出码

经过一系列初始化过程后，用户程序就成功在 StarryX 上运行起来。

3. 进程管理

3.1. 整体架构

系统的任务管理基础由 `axtask` 模块提供。`axtask` 本身不关心“进程”或“线程”等高级概念，它只负责定义和管理最基本的可调度实体（`Task`）。

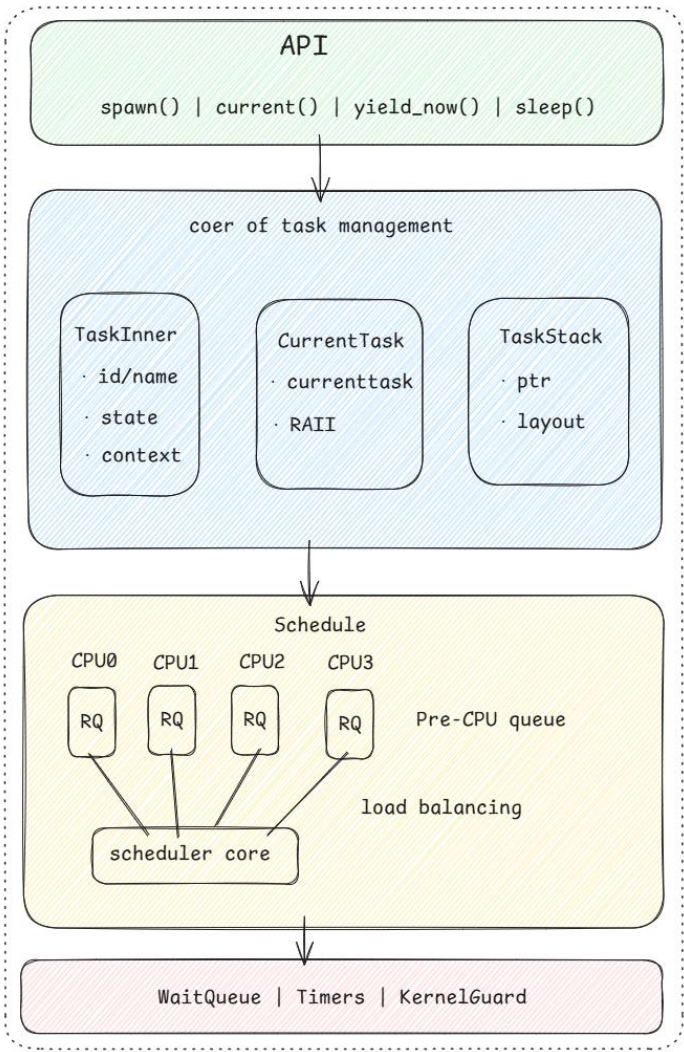


图 3.1 进程管理架构

`axtask` 的核心架构由三个关键组件构成：

1. **Task**: 这是内核中可独立调度的最小执行单元。它包含了任务运行所需的最基本信息，如任务状态、上下文（寄存器）、内核栈等。
2. **runqueue (运行队列)**: 这是一个存放所有处于就绪 (`Ready`) 状态任务的队列。当一个任务被创建，或者从阻塞状态被唤醒后，它就会被放入运行队列，等待调度器选择它并投入运行。为了在多核环境下实现高效调度，系统通常会

为每个 CPU 核心维护一个独立的 runqueue，以避免核间锁竞争。

3. waitqueue (等待队列): 这是一个用于管理阻塞（Blocked）状态任务的数据结构。当一个任务需要等待某个事件（例如，等待 I/O 操作完成、等待一个锁被释放、等待一个信号量）时，它会从运行队列中移除，并被放入与该事件关联的等待队列中。当事件发生时，内核会从等待队列中唤醒相应的任务，并将其重新放回运行队列。这是一种高效的、事件驱动的同步机制，避免了任务的忙等待。

3.2. 任务结构

每个可调度实体在内核中都由一个核心的数据结构来描述，通常被称为任务控制块（TCB）。在 axtask 中，这个结构（我们称之为 Task 结构体）包含了管理一个任务所需的所有信息：

```
1. pub struct TaskInner {  
2.     id: TaskId,           // 唯一标识符  
3.     name: String,         // 任务名称  
4.     state: TaskState,     // 任务状态  
5.     cpumask: AxCpuMask,   // CPU 亲和性  
6.     ctx: TaskContext,     // 上下文信息  
7.     kstack: TaskStack,    // 内核栈  
8.     task_ext: AxTaskExt,  // 扩展数据  
9.     // ...其他字段  
10. }
```

身份标识: 每个任务都有唯一的 TaskID 和可读的名称。TaskID 是 64 位整数，通过原子计数器生成，确保唯一性。任务名称主要用于调试和日志输出。

状态管理: 任务状态使用原子变量存储，支持无锁的状态查询和转换。状态转换必须遵循严格的规则，例如只有 Ready 状态的任务才能转换为 Running 状态。

执行上下文: 包含 CPU 寄存器状态、栈指针等信息。上下文切换时，当前任务的上下文会被保存，新任务的上下文会被加载。

内存管理: 每个任务有独立的内核栈，用于存储局部变量、函数调用信息等。栈大小可以配置，默认为 4KB。

CPU 亲和性: 指定任务可以在哪些 CPU 核心上运行。这对于性能优化和实时性保证非常重要。

扩展数据: 支持用户自定义的扩展数据，通过类型安全的接口访问。这使得不同的应用可以为任务添加特定的属性。

3.3. 任务调度

任务调度决定了哪个就绪的任务可以获得 CPU 的使用权。这个决策由 axsched (Scheduler) 模块负责。

```
1. cfg_if::cfg_if! {
2.     if #[cfg(feature = "sched_rr")] {
3.         type Scheduler = scheduler::RRScheduler<TaskInner,
MAX_TIME_SLICE>;
4.     } else if #[cfg(feature = "sched_cfs")] {
5.         type Scheduler = scheduler::CFScheduler<TaskInner>;
6.     } else if #[cfg(feature = "sched_dynamic")] {
7.         type Scheduler = scheduler::DynamicScheduler<TaskInner>;
8.     } else {
9.         type Scheduler = scheduler::FifoScheduler<TaskInner>;
10.    }
11. }
```

axsched: 这是一个可插拔的调度器模块。它可以实现不同的调度策略，例如:

1. FIFO (先进先出): 按任务进入 runqueue 的顺序进行调度。
2. Round-Robin (轮询): 每个任务被分配一个固定的时间片，时间片用完后，即使任务没执行完，也会被放回 runqueue 的末尾，切换到下一个任务。
3. CFS: 根据任务的优先级和已运行时间，动态计算权重，选择最“需要”运行的任务，力求公平。

```
1. // 每个 CPU 都有独立的运行队列
2. percpu_static! {
3.     RUN_QUEUE: LazyInit<AxRunQueue> = LazyInit::new(),
4.     // ...
5. }
6.
7. // 全局运行队列数组，按 CPU ID 索引
8. static mut RUN_QUEUES: [MaybeUninit<&'static mut AxRunQueue>;
```

```
axconfig::SMP] =  
9. [ARRAY_REPEAT_VALUE; axconfig::SMP];
```

多核调度：在多核 CPU 上，调度变得更加复杂。StarryX 采用 Per-CPU runqueue 的设计：

1. 独立队列：每个 CPU 核心拥有自己独立的 runqueue。当需要调度时，CPU 只在自己的队列里选择任务，这极大地减少了多个 CPU 同时访问同一个队列而产生的锁争用，提高了调度效率。
2. 任务迁移与负载均衡：这种设计引入了新的问题：如果一个 CPU 的 runqueue 很长（负载高），而另一个 CPU 的 runqueue 是空的（负载低），系统效率就会下降。因此，需要一个负载均衡器周期性地检查各个 CPU 的负载，并将任务从繁忙的 CPU“偷”到空闲的 CPU 上，以实现全局的负载均衡。
3. 任务亲和性 (Affinity): 系统也支持将某个任务“绑定”到特定的 CPU 上，这对于需要利用 CPU 缓存的性能敏感型应用非常重要。

```
1. pub struct TaskInner {  
2.     // ...  
3.     /// CPU affinity mask.  
4.     cpumask: SpinNoIrq<AxCpuMask>,  
5.     // ...  
6. }  
7.  
8. // 设置任务亲和性  
9. pub fn set_affinity(task: &AxTaskRef, cpumask: AxCpuMask) -> bool {  
10.     if cpumask.is_empty() {  
11.         false  
12.     } else {  
13.         task.set_cpumask(cpumask);  
14.         // 设置亲和性后检查是否需要迁移  
15.         #[cfg(feature = "smp")]  
16.         migrate_current(task.clone());  
17.         true  
18.     }  
19. }
```

3.4. 任务扩展

`axtask` 提供的只是一个最小化的执行单元。`StarryX` 通过 `xprocess` 模块, 将这个基础的 `Task` 扩展为现代操作系统中常见的线程 (`Thread`) 和进程 (`Process`) 模型。这个扩展过程是一个优雅的“层层包装”:

1. `axtask::Task` (内核任务): 这是最底层的“原子”执行流, 只知道如何运行代码。

2. `xthread` (内核线程): `xprocess` 模块首先在 `axtask::Task` 的基础上进行扩展, 创建出内核线程的概念。一个 `xthread` 本质上是一个 `axtask::Task`, 但通过 `Task` 的扩展指针, 它额外关联了属于同一个进程的其他信息, 例如信号处理器、线程局部存储 (TLS) 等。

```
1. pub struct Thread {  
2.     tid: Pid,  
3.     process: Arc<Process>,  
4.     data: Box<dyn Any + Send + Sync>,  
5. }
```

3. `xprocess` (内核进程): `xprocess` 模块再将多个 `xthread` 组织在一起, 形成一个内核进程。进程是资源分配的单位。一个进程包含:

1. 一个或多个内核线程 (`xthread`)。
2. 一个独立的虚拟地址空间 (由 `xvma` 模块管理)。
3. 一张文件描述符表。
4. 一组信号处理规则 (`xsignal`)。
5. 其他共享资源。

这个模型清晰地区分了调度和资源管理: 调度的基本单位是线程 (`xthread`, 其核心是 `axtask::Task`)。资源分配和隔离的基本单位是进程 (`xprocess`)。

```
1. pub struct Process {  
2.     pid: Pid,  
3.     is_zombie: AtomicBool,  
4.     pub(crate) tg: SpinNoIrq<ThreadGroup>,  
5.  
6.     pub(crate) data: Box<dyn Any + Send + Sync>,  
7.     children: SpinNoIrq<StrongMap<Pid, Arc<Process>>>,&br/>8.     parent: SpinNoIrq<Weak<Process>>,&br/>9. }
```



```
10.     group: SpinNoIrq<Arc<ProcessGroup>>,
11. }
```

最终，xprocess 和 xthread 模块通过 xapi（系统调用接口）向用户空间暴露了我们所熟悉的 fork(), exec(), pthread_create() 等接口，从而为应用程序提供了功能完备的进程和线程模型。

3.5. 进程通信

Starry X 的 IPC 模块实现了经典 UNIX System V IPC 机制,包括消息队列、信号量和共享内存。该模块的设计采用了清晰的 **前端-后端** 分离架构,将系统调用接口与核心逻辑实现分离,提高了代码的模块化程度、可读性和可维护性。整个模块围绕一个中心化的 IpcManager 进行管理,并利用 Mutex 和 Arc 确保了在多任务并发环境下的线程安全。

1、共享内存

System V 共享内存管理系统,采用类 Linux 内核的设计思路。可以使多个进程的不同虚拟地址映射到相同的物理内存页面,实现数据共享。

StarryX 的共享内存原理在于,它将一个逻辑 shmid 映射到一个由原子引用计数(Arc)管理的单一物理内存实例(SharedPages)上,该实例被懒惰分配并可由多个进程映射到各自独立的虚拟地址空间,而其最终的物理回收则被精确地延迟到最后一个附加进程分离且该内存段已被预先标记为删除(IPC_RMID)这两个条件同时满足的时刻。

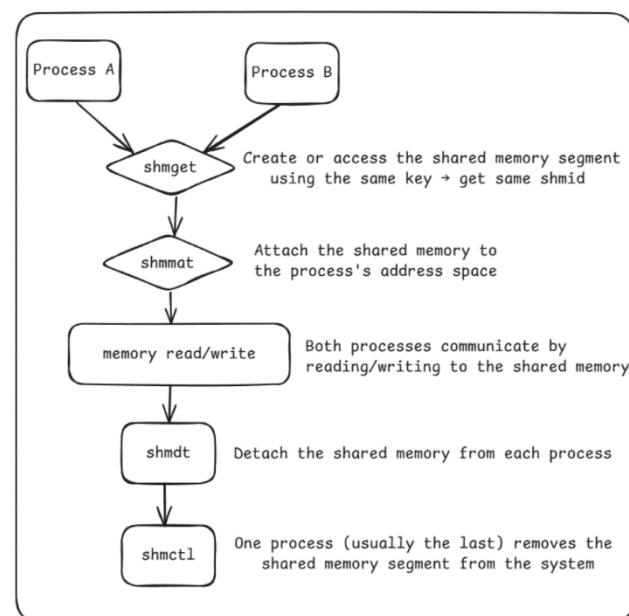


图 3.2 Shm 工作流程图

```

1. pub struct ShmManager {
2.     index: BTreeMap<i32, i32>,
3.     segments: BTreeMap<i32, Arc<Mutex<ShmSegment>>>,
4.     pid_shmid_vaddr: BTreeMap<Pid, BiBTreeMap<i32, VirtAddr>>,
5.     id_generator: Mutex<IpcidGenerator>,
6. }

```

相关系统调用实现：

sys_shmget：创建或获取共享内存段，验证参数和系统限制

sys_shmat：将共享内存段映射到进程地址空间

sys_shmdt：从进程地址空间分离共享内存段

sys_shmctl：执行控制操作，如查询状态、设置参数、删除段等

2、消息队列

消息队列是一种进程间通信机制，允许进程通过发送和接收消息来进行异步通信。本实现基于 System V IPC 标准，提供了内核级别的消息传递服务。

在进行消息队列通信时，生产者进程和消费者进程可以独立的运行，不需要同时在线或者直接知道对方存在，通过消息队列作为中介实现通信。这就意味着发送者在调用 `msgsnd()` 后立即返回，无需等待接收者处理，而接收者可以在任意时刻通过 `msgrcv()` 获取消息。发送的消息到达队列后会存储在内核的 `VecDeque<Message>` 中，即使发送进程退出了，消息依然存在，直到被接收者消费。

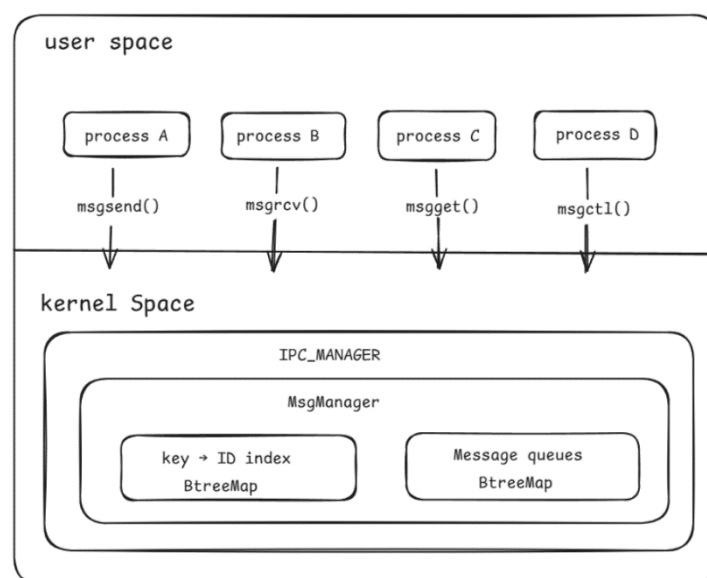


图 3.3 消息队列框架图


```
1. pub struct MsgManager {
2.     index: BTreeMap<i32, i32>,
3.     queues: BTreeMap<i32, Arc<Mutex<MsgQueue>>>>,
4.     id_generator: Mutex<IpcidGenerator>,
5. }
```

相关系统调用实现：

sys_msgget：根据 key 创建或获取消息队列，处理权限和系统限制

sys_msgsnd：向指定队列发送消息，处理用户空间数据复制和权限检查

sys_msgrcv：从指定队列接收消息，支持消息类型过滤

sys_msgctl：执行控制操作，如查询状态、设置参数、删除队列等

3、信号量

本系统实现的 System V 信号量是 linux 中一种传统的进程间通信机制，主要用于在多个进程之间实现同步与互斥，防止对共享资源的冲突访问。

它本质上是一个计数器，可以控制同时访问某个资源的进程数量。当进程需要访问共享资源时，会对信号量执行 P 操作（减 1），如果信号量值为 0 则进程阻塞等待；当进程释放资源时，执行 V 操作（加 1），并唤醒等待的进程。信号量系统支持批量操作多个信号量，提供原子性保证，还具有 SEM_UNDO 机制确保进程异常退出时自动撤销操作，避免死锁。它广泛应用于生产者-消费者模型、资源池管理、互斥锁实现等场景，是构建可靠并发系统的重要工具。

```
1. pub struct SemManager {
2.     index: BTreeMap<i32, i32>,
3.     semsets: BTreeMap<i32, Arc<Mutex<SemSet>>>>,
4.     pid_undo: BTreeMap<Pid, Vec<SemUndo>>>,
5.     id_generator: Mutex<IpcidGenerator>,
6.     total_semaphores: usize,
7. }
```

相关系统调用实现：

sys_semget：创建或获取信号量集，验证参数

sys_semop：执行信号量操作，支持阻塞和非阻塞模式

sys_semctl：执行多种控制操作，如查询/设置值、删除信号量集等

4. 内存管理

4.1. 整体架构

StarryX 的内存管理模块主要由内存分配模块和虚拟内存管理模块组成，它们的内核基础功能分别由 arceos 的 `axalloc` 模块和 `axmm` 模块提供。考虑到系统性能需求，StarryX 使用单一页表架构，内核与用户共享地址空间，无需频繁切换根页表产生开销。对于内存分配模块，`axalloc` 实现了一个全局内存分配器，支持多种内存分配算法，并提供 `api` 实现灵活内存分配；对于虚拟内存管理模块，`axmm` 实现了 `'AddrSpace'` 管理任务地址空间，宏内核在其基础上扩展了进程地址空间。另外我们实现了写时复制、内存懒分配等高级机制，将用户空间访问解耦为组件提供复用。

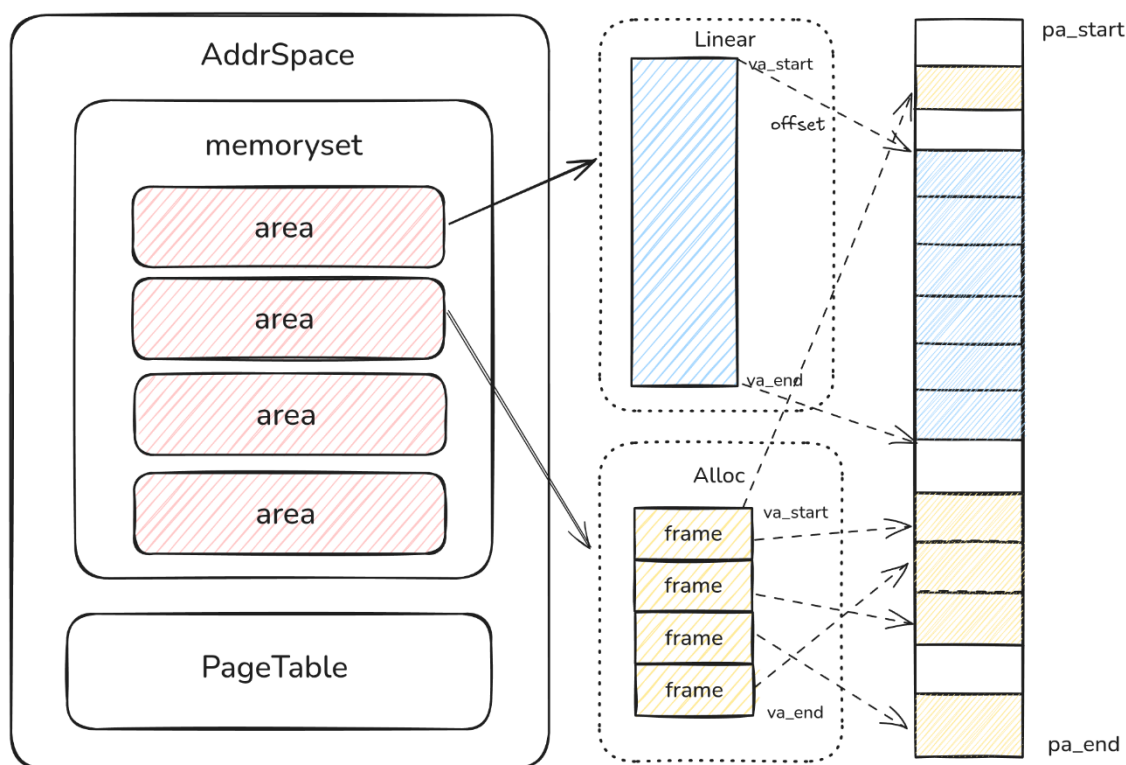


图 4.1 axmm 系统架构图

4.2. 内存分配器

内存分配的主要逻辑在 `axalloc` 模块中实现，其核心为 `GlobalAllocator` 结构体：

```
1. pub struct GlobalAllocator {  
2.     balloc: SpinNoIrq<DefaultByteAllocator>,  
3.     palloc: SpinNoIrq<BitmapPageAllocator<PAGE_SIZE>>,  
4. }
```

GlobalAllocator 由两部分组成，palloc 使用位图分配器管理物理页的分配，而 balloc 提供了字节范围的分配接口，当用户申请内存分配时，先分配 balloc 分配器的内存，若 balloc 分配器内存不足，则从 palloc 分配器分配内存。其中 balloc 实现了多种内存分配算法，包括 Buddy、Slab 和 Tlsf，可以通过 feature 自由选择：

```
1. cfg_if::cfg_if! {  
2.     if #[cfg(feature = "slab")] {  
3.         /// The default byte allocator.  
4.         pub type DefaultByteAllocator = allocator::SlabByteAllocator;  
5.     } else if #[cfg(feature = "buddy")] {  
6.         /// The default byte allocator.  
7.         pub type DefaultByteAllocator = allocator::BuddyByteAllocator;  
8.     } else if #[cfg(feature = "tlsf")] {  
9.         /// The default byte allocator.  
10.        pub type DefaultByteAllocator = allocator::TlsfByteAllocator;  
11.    }  
12. }
```

axalloc 为内核提供内存分配的接口的同时，StarryX 为该模块新添加了内存回收接口，通过使用 crate_interface 组件解决循环依赖问题，当内核内存不足时，可以回收别的内核功能实现的内存，比如页缓存：

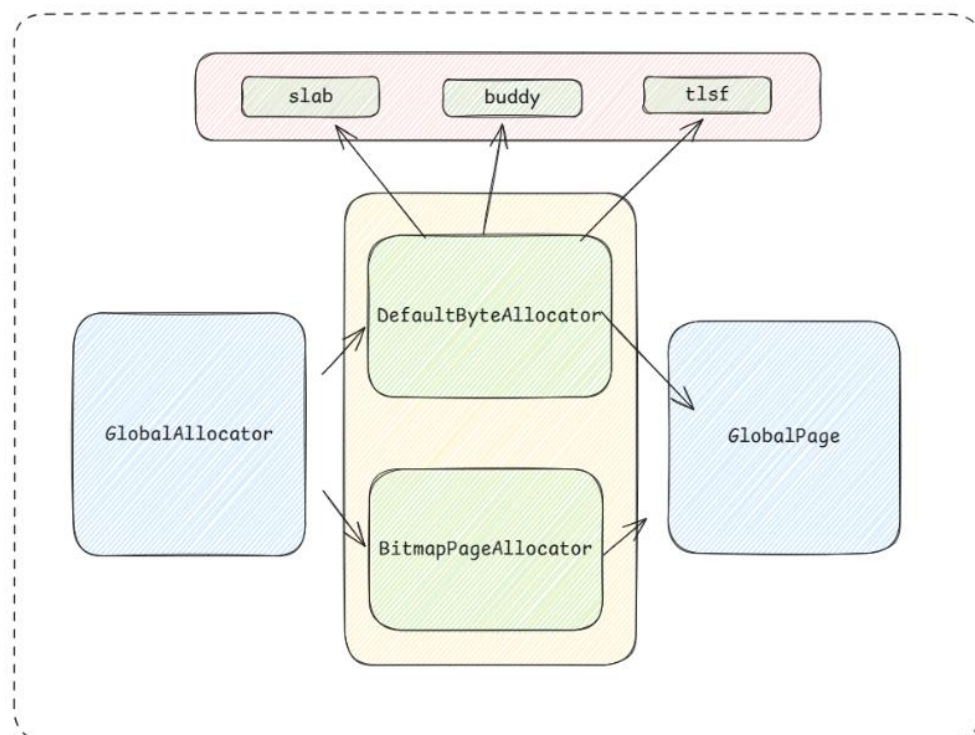


图 4.2 axalloc 框架图

```

1. #[crate_interface::def_interface]
2. pub trait AxAllocIf {
3.     fn evict_cache(num_pages: usize) -> AllocResult;
4. }

```

4.3. 地址空间管理

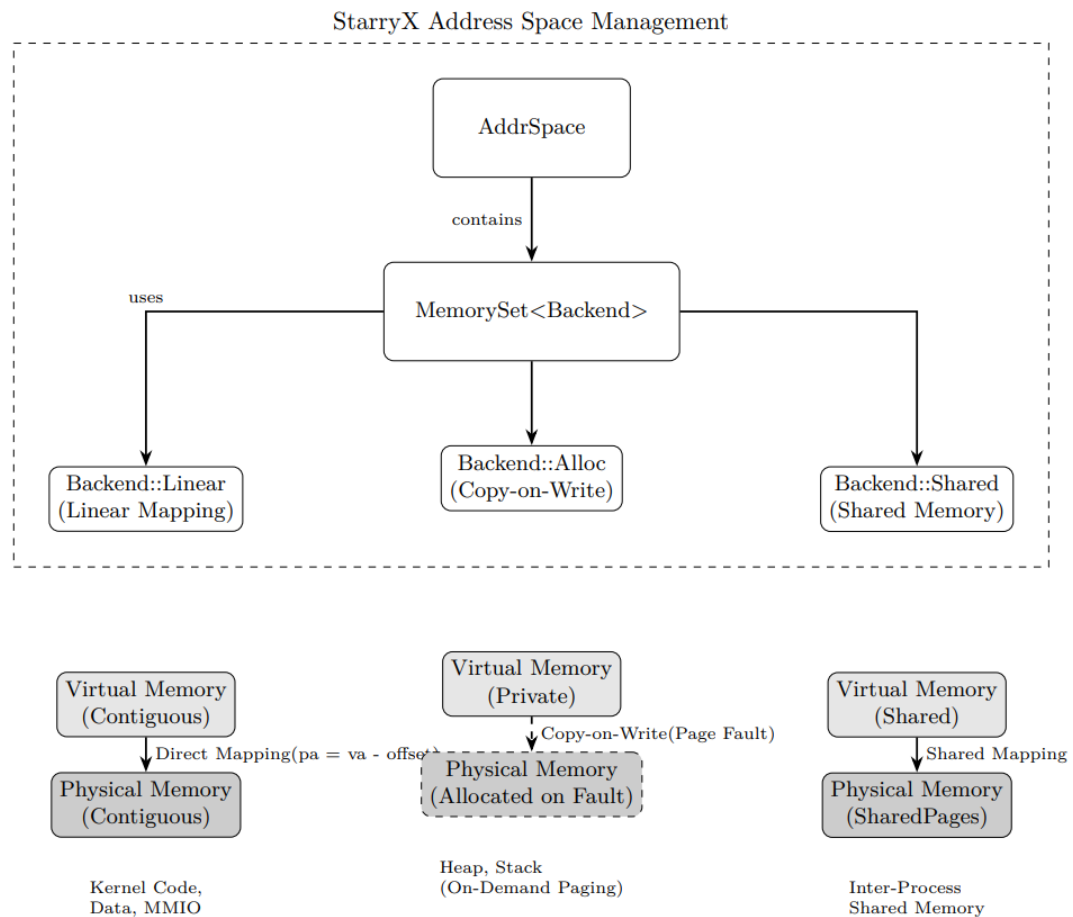


图 4.3 地址空间框架图

StarryX 的地址空间通过`axmm`模块的`AddrSpace`结构体管理，其包括三个字段，`va\_range` 管理虚拟地址范围、`areas` 管理具体的内存区域、`pt` 为该地址空间下的虚拟页表：

```

1. /// The virtual memory address space.
2. pub struct AddrSpace {
3.     va_range: VirtAddrRange,
4.     areas: MemorySet<Backend>,
5.     pt: PageTable,
6. }

```

其中 `areas` 为内存区域集合 `MemorySet`，`MemorySet` 通过 B 树管理该地址空间的内存区域。

```
1. pub struct MemorySet<B: MappingBackend> {  
2.     areas: BTreeMap<B::Addr, MemoryArea<B>>,  
3. }
```

对于每个内存区域，有特定的映射方式，每个特定映射都需要实现以下接口：

```
1. pub trait MappingBackend {  
2.     /// What to do when mapping a region within the area with the given flags.  
3.     fn map(  
4.         &self,  
5.         start: Self::Addr,  
6.         size: usize,  
7.         flags: Self::Flags,  
8.         page_table: &mut Self::PageTable,  
9.     ) -> bool;  
10.  
11.    /// What to do when unmaping a memory region within the area.  
12.    fn unmap(&self, start: Self::Addr, size: usize, page_table: &mut Self::PageTable)  
13.  
14.    /// What to do when changing access flags.  
15.    fn protect(  
16.        &self,  
17.        start: Self::Addr,  
18.        size: usize,  
19.        new_flags: Self::Flags,  
20.        page_table: &mut Self::PageTable,  
21.    ) -> bool;  
22. }
```

`axmm` 一共实现了三种映射方式，每种映射方式分别应用于不同场景：

1. 线性映射（`linear`）：线性映射直接将虚拟内存按照偏移线性映射到特定的物理内存范围，线性映射一般常用于映射内核代码段、数据段以及设备 MMIO 区域

2. 动态映射（`alloc`）：`alloc` 映射动态分配物理内存，通过 `alloc` 可以实现写时复制和懒分配机制，只有当程序访问该 VMA 内的某个地址并触发缺页异常时，才动态分配一个物理页帧并建立映射。这是实现按需分页的基础，常用于进程的堆和栈。

3. 共享映射（shared）：可用于实现进程间的共享内存(MAP_SHARED)和 System V 共享内存机制。它在创建时就分配好全部所需的物理页，并由一个共享对象 (SharedPages) 持有。其他进程映射同一块共享内存时，会复用这些已分配的物理页，从而实现对同一物理内存的并发读写。

4.4. 延迟分配技术

延迟分配技术是操作系统重要的内存技术，其核心思想是不提前分配物理内存，而是等程序真正访问（触发缺页异常）时，再去分配并建立映射，StarryX 在 axmm 中扩展实现了懒分配和写时复制两种重要机制，极大减少了进程复制时的内存复制开销。

4.4.1. 懒分配

我们在内存映射（mmap）、程序中断点（brk）、用户栈（stack）的分配过程中使用了懒分配技术，其实现核心是内存区域的 alloc 映射，alloc 映射内部维护一个关键元数据 populate，其可以通过调用者在创建内存区域时自由指定。

```
1. pub enum {
2.     Alloc {
3.         /// Whether to populate the physical frames when creating the mapping.
4.         populate: bool,
5.     },
6. }
```

对于 populate 为 true 的情况，直接分配物理内存并建立分页映射；对于 populate 为 false 的情况将延迟分配内存，由于未在页表建立映射，当用户读取对应页面的数据时将触发缺页异常，从而使得访存行为被内存捕获，此时会执行页面错误处理程序，当从 MemorySet 中找到对应内存范围后执行特定函数，检查 populate 元数据并执行分配和映射：

```
1. pub(crate) fn handle_page_fault_alloc(
2.     vaddr: VirtAddr,
3.     orig_flags: MappingFlags,
4.     pt: &mut PageTable,
5.     populate: bool,
6.     align: PageSize,
7. ) -> bool {
8.     if populate {
9.         false
10.    } else if let Some(frame) = alloc_frame(true, align) {
```

```

11.         pt.map(vaddr, frame, PageSize::Size4K, orig_flags)
12.         .map(|tlb| tlb.flush())
13.         .is_ok()
14.     } else {
15.         false
16.     }
17. }

```

特别地, `mmap` 除了会建立直接的内存映射, 还会创建文件映射(`MAP_FILE`), 对于这一类映射, 将会在内存管理模块 `axmm` 和文件系统 `axfs` 建立练习, 造成模块耦合且在 `arceos` 引入了宏内核内容, 为了避免这种情况我们实现了模块解耦的虚拟内存管理模块 (`xvma`), 通过其我们实现了文件页的懒分配, 有效解决了这一种情况, 我们将在第七章深入介绍。

4.4.2. 写时复制

宏内核的以 `clone()` 为代表的进程复制操作中, 在创建新的子进程时需要将原有的内存信息完整地拷贝一份, 这一行为通常会耗费大量的内存空间, 为内核带来巨大的内存开销负担, 为了尽量避免进程复制时的内存开销, 我们引入了写时复制 (`copy-on-write`) 技术。

对于 COW 的实现, 我们在全局维护一张物理页帧表(Frame Table), 每一个物理页帧对应一个 `FrameInfo` 结构体, `FrameInfo` 结构体内部维护一个引用计数, 当 `StarryX` 处理 `clone` 时会调用 `axmm` 的 `try_clone()`, `try_clone()` 将会对被复制的每一个内存区域进行处理, 去除写标志位并对对应的 `Frameinfo` 增加引用计数, 只有 `Frameinfo` 的引用计数为 0 时才会被真正释放。当用户真正写入时由于该页表项未设置 W 位, 该操作将被操作系统捕获, 进行实际的页面复制行为。

```

1. pub(crate) struct FrameRefTable {
2.     data: Box<[FrameInfo; MAX_FRAME_NUM]>,
3. }
4. pub(crate) struct FrameInfo {
5.     ref_count: AtomicUsize,
6. }

```

```

1. #[cfg(feature = "cow")]
2. fn handle_cow_fault(
3.     vaddr: VirtAddr,
4.     paddr: PhysAddr,
5.     flags: MappingFlags,
6.     align: PageSize,

```



```

7.     pt: &mut PageTable,
8. ) -> bool {
9.     match frame_table().ref_count(paddr) {
10.         0 => unreachable!(),
11.         // There is only one AddrSpace reference to the page,
12.         // so there is no need to copy it.
13.         1 => pt.protect(vaddr, flags).map(|(_, tlb)| tlb.flush()).is_ok(),
14.         // Allocates the new page and copies the contents of the original page,
15.         // remapping the virtual address to the physical address of the new page.
16.         2.. => match alloc_frame(false, align) {
17.             Some(new_frame) => {
18.                 unsafe {
19.                     core::ptr::copy_nonoverlapping(
20.                         phys_to_virt(paddr).as_ptr(),
21.                         phys_to_virt(new_frame).as_mut_ptr(),
22.                         align.into(),
23.                     )
24.                 };
25.
26.                 dealloc_frame(paddr, align);
27.
28.                 pt.remap(vaddr, new_frame, flags)
29.                     .map(|(_, tlb)| {
30.                         tlb.flush();
31.                     })
32.                     .is_ok()
33.             }
34.             None => false,
35.         },
36.     }
37. }

```

4.5. 用户地址访问

xuspace 组件是 StarryX 中负责用户地址空间访问的核心模块，它为内核提供了安全、统一的用户空间内存访问接口，其封装了用户态地址访问的复杂性，确保内核在访问用户空间数据时的安全性和正确性。

在设计之初，xuspace 与 xcore 和 xapi 紧密耦合，由于初期时内存相关机制尚未实现，解耦于 StarryX 的组件（比如 xsignal）可以将用户指针转化为裸指针进行访问，但是这样的访问存在许多问题：

1. 无法判断其合法性，无法安全访问用户地址
2. 引入了大量对裸指针的 unsafe 操作
3. 实现 cow 后可能导致在内核态发生缺页异常而发生致命错误

当实现内存延迟分配机制后，解耦于 xcore 的组件无法再正常访问用户地址空间，因此我们将 xuspace 从 xcore 中解耦成为一个独立组件，并提供抽象接口使其可以被其他系统所复用。我们将在第七章详细介绍这一组件。

4.6. 页面异常处理

页面异常的处理方面，StarryX 在 ArceOS 的基础上增加了判断：如果是用户态程序访问内存空间或者是内核态程序访问用户内存空间时触发的缺页异常，才会调用用户程序的 handle_page_fault 函数，也就是 ArceOS 的 AddrSpace 结构体提供的页面错误处理函数处理缺页异常，该函数会根据虚拟地址的映射关系，调用全局内存分配器进行物理页的分配和映射。如果 AddrSpace 结构体不能成功处理该异常，则会打印错误信息并终止当前任务，并发送 SIGSEGV 信号。

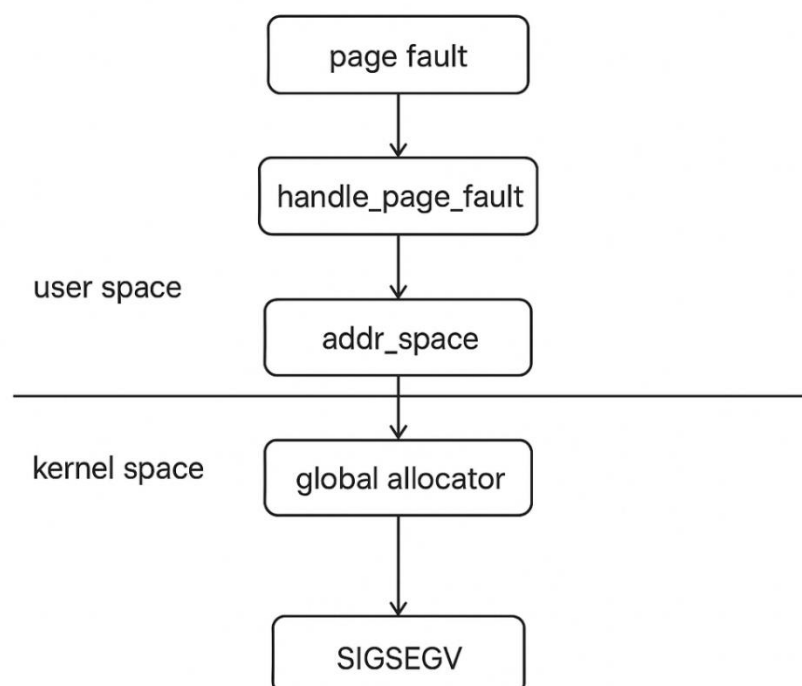


图 4.4 页面异常处理流程

5. 文件系统

5.1. 整体架构

StarryX 的文件系统架构可分为三层，最底层为 arceos 中支撑 axfs 运行的虚拟文件系统 axfs-vfs，在其之上是 axfs，其实现了 axfs-vfs 的具体文件系统实例，如 ext4、vfat，最上层为 X Core，其对磁盘文件和宏内核的抽象文件进行封装，通过统一的 trait 进行文件操作抽象。

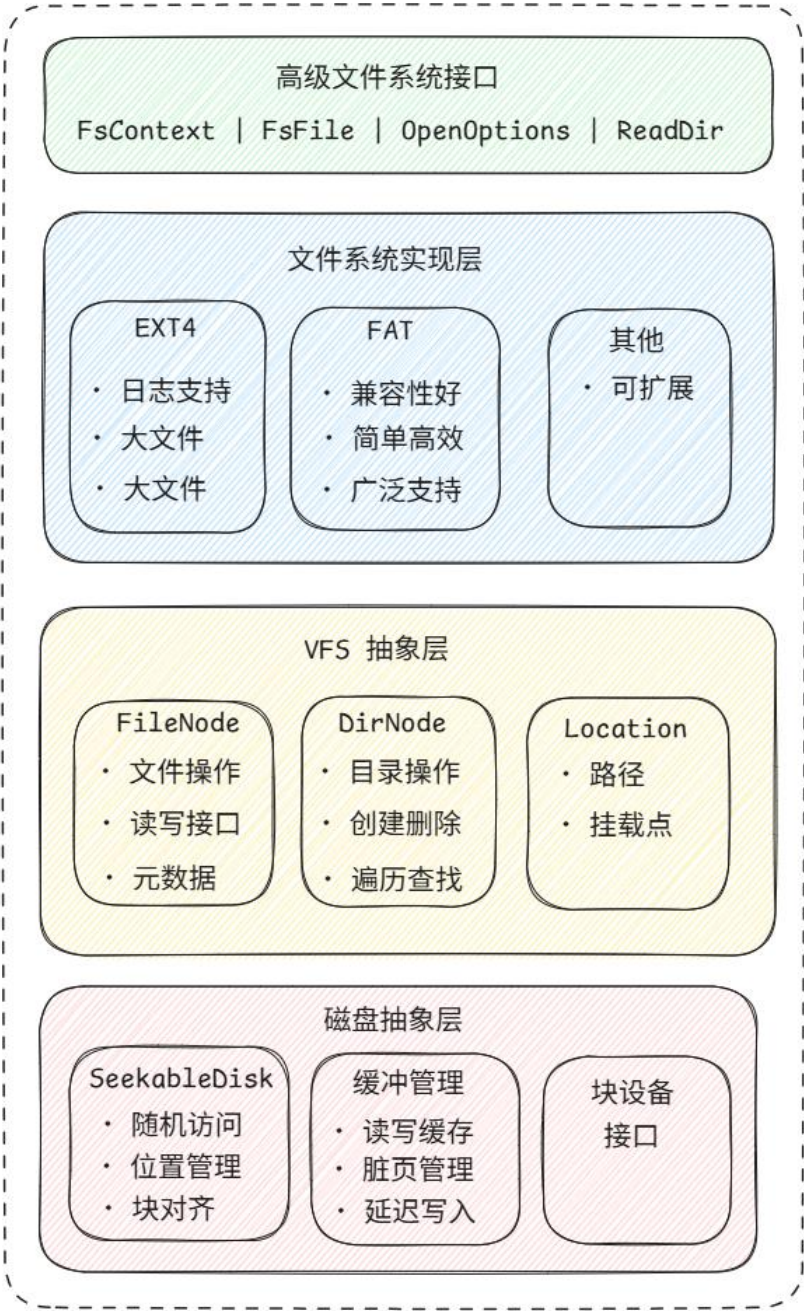


图 5.1 文件系统架构图

5.2. 虚拟文件系统

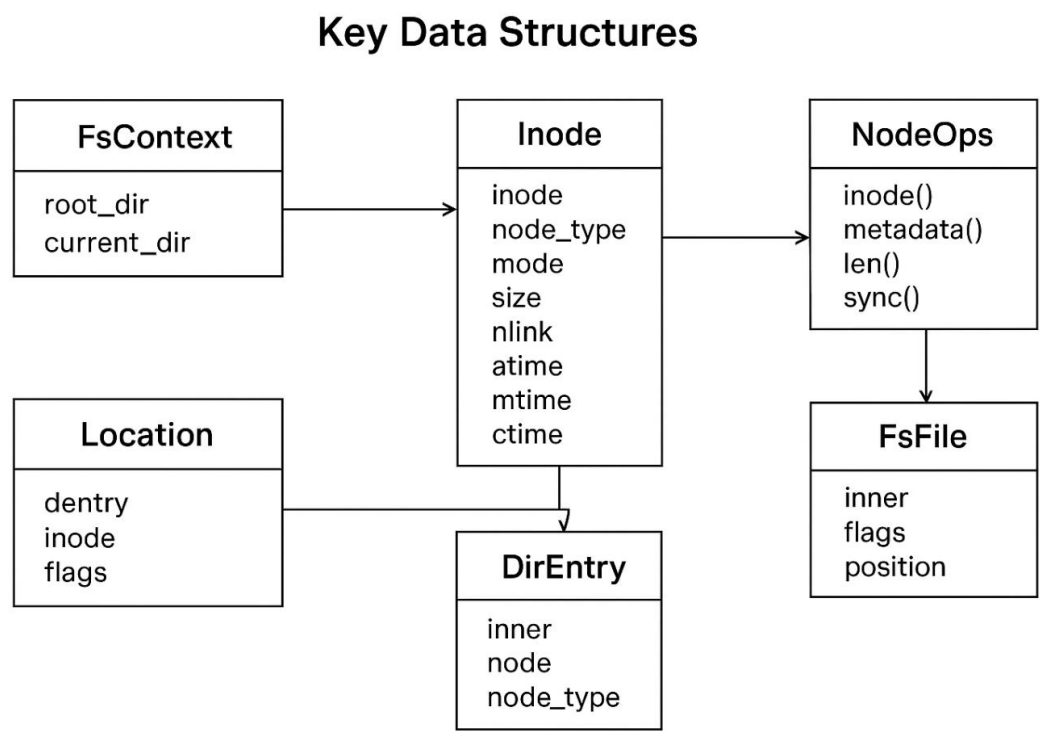


图 5.2 虚拟文件系统架构图

StarryX 采用了经典的、受 Unix 启发的 VFS 设计，其核心是围绕一组标准对象来组织的：

```
1. /// 文件系统上下文：管理挂载点和路径解析
2. pub struct FsContext<M> {
3.     root_dir: Location<M>,      // 根目录
4.     current_dir: Location<M>,   // 当前工作目录
5. }
```

`fs_context`: 代表一个已挂载的文件系统实例。它包含了该文件系统的全局信息，如文件系统类型（FAT32/Ext4）、块大小、总空间、根节点的 `inode` 等。在 Unix 中，这被称为超级块。

```
1. /// Inode 元数据：文件的核心信息
2. #[derive(Clone, Debug)]
3. pub struct Metadata {
4.     pub inode: u64,              // 唯一 inode 编号
5.     pub node_type: NodeType,    // 文件类型（文件/目录/符号链接）
6.     pub mode: NodePermission,   // 权限模式
```

```

7.     pub size: u64,                // 文件大小
8.     pub nlink: u64,              // 硬链接数
9.     pub atime: Duration,         // 访问时间
10.    pub mtime: Duration,         // 修改时间
11.    pub ctime: Duration,         // 状态改变时间
12. }
13.
14. /// 节点操作：统一的文件/目录接口
15. pub trait NodeOps<M>: Send + Sync {
16.     fn inode(&self) -> u64;
17.     fn metadata(&self) -> VfsResult<Metadata>;
18.     fn len(&self) -> VfsResult<u64>;
19.     fn sync(&self, data_only: bool) -> VfsResult<()>;
20. }

```

Inode: 代表一个文件或目录。它是文件的核心元数据，包含了文件的类型（普通文件、目录、符号链接等）、权限、大小、创建/修改时间，以及最重要的——指向存储文件内容的数据块指针。每个文件或目录在文件系统中都有一个唯一的 **inode**。

```

1. /// 目录项：文件名与 inode 的绑定
2. pub struct DirEntry<M>(Arc<Inner<M>>);
3.
4. struct Inner<M> {
5.     node: Node<M>,                // 实际的 inode
6.     node_type: NodeType,          // 节点类型缓存
7.     reference: Reference<M>,     // 父目录引用
8. }
9.
10. /// 引用关系：(父目录, 文件名) 的映射
11. pub struct Reference<M> {
12.     parent: Option<DirEntry<M>>, // 父目录
13.     name: String,                 // 文件名
14. }

```

dentry: 代表一个目录中的条目，它是一个将文件名（如 "myfile.txt"）与 **inode** 编号进行绑定的键值对。目录本身的内容就是一张 **dentry** 列表。VFS 通过逐级解析路径中的 **dentry** 来实现路径查找（Path Lookup），例如，查找 /home/user 就

是在根目录 / 的 dentry 列表中找到"home", 再在 "home" 对应的 inode (一个目录) 中找到 "user" 的 dentry。

```
1. /// 文件句柄: 进程特定的文件访问状态
2. pub struct FsFile<M> {
3.     inner: Location<M>,          // 指向 dentry 和 inode
4.     flags: FileFlags,           // 打开标志 (读/写/追加)
5.     pub position: u64,          // 当前读写位置
6. }
```

location: 代表一个已打开的文件。当进程调用 open()后, 内核会创建一个 file 对象。这个对象包含了指向对应 dentry 和 inode 的指针, 以及该进程对此文件的特定状态, 比如读写模式 (只读/读写)、以及当前读写位置的偏移量。这允许多个进程同时打开同一个文件, 但各自拥有独立的读写位置。

5.3. 文件系统实例

在 StarryX 中, 我们集成了对 VFAT (FAT16/FAT32) 和 Ext4 这两种广泛使用的文件系统的支持, 并通过自适应的封装确保它们能在多线程环境中安全高效地运行。

·VFAT (FAT16/FAT32) 支持

```
1. pub struct FatFilesystem<M> {
2.     inner: Mutex<M, FatFilesystemInner>, // 全局互斥锁
3.     root_dir: Mutex<M, Option<DirEntry<M>>>,
4. }
5.
6. impl<M: RawMutex> FatFilesystem<M> {
7.     pub(crate) fn lock(&self) -> MutexGuard<M, FatFilesystemInner> {
8.         self.inner.lock() // 获取文件系统锁
9.     }
10. }
```

我们基于 rust-fatfs 库实现了对 VFAT 文件系统的支持。由于原库主要为单线程环境设计, 且缺少部分关键功能, 我们进行了两项主要改进:

1. 功能扩展: 我们为 rust-fatfs 手动添加了缺失的接口, 例如获取文件修改时间、读取目录元数据等, 使其功能更加完备。

2. 并发安全: 为了解决原库难以扩展到多线程的问题, 我们在全局的

FileSystem 实例上添加了互斥锁。同时，我们设计了 `FsRef<T>` 包装类型，它要求在访问文件 (File) 或目录(Dir) 对象前必须先获取 FileSystem 的锁，从而从机制上保证了并发访问的安全性。

Ext4 支持

```
1. pub(crate) struct Ext4Disk(AxBlockDevice);
2.
3. impl BlockDevice for Ext4Disk {
4.     fn read_blocks(&mut self, block_id: u64, buf: &mut [u8]) -> Ext4Result<usize> {
5.         let mut block_buf = [0u8; EXT4_DEV_BSIZE];
6.         for (i, block) in buf.chunks_mut(EXT4_DEV_BSIZE).enumerate() {
7.             self.0
8.                 .read_block(block_id + i as u64, &mut block_buf)
9.                 .map_err(|_| Ext4Error::new(EIO as _, None))?;
10.            block.copy_from_slice(&block_buf);
11.        }
12.        Ok(buf.len())
13.    }
14.
15.    fn write_blocks(&mut self, block_id: u64, buf: &[u8]) -> Ext4Result<usize> {
16.        let mut block_buf = [0u8; EXT4_DEV_BSIZE];
17.        for (i, block) in buf.chunks(EXT4_DEV_BSIZE).enumerate() {
18.            block_buf.copy_from_slice(block);
19.            self.0
20.                .write_block(block_id + i as u64, &block_buf)
21.                .map_err(|_| Ext4Error::new(EIO as _, None))?;
22.        }
23.        Ok(buf.len())
24.    }
25. }
```

我们采用 `lwext4` C 库作为 Ext4 的底层实现。然而，社区已有的 Rust 封装层 `lwext4_rust` 将文件系统操作与路径解析混合在一起，不符合我们的设计理念。因此，我们采取了以下措施：

1. 重写封装层：我们完全重写了 `lwext4_rust`，提供了一套纯粹基于 `inode` 的文件系统操作接口，将文件操作与路径解析清晰地分离。

2. 移除全局依赖：原 `lwext4` 的部分操作（如 `read`、`write`）强依赖于全局变量，导致无法同时创建多个文件系统实例。我们在新的 Rust 封装层中重构了这

些接口，消除了对全局状态的依赖，从而允许在系统中同时挂载和操作多个独立的 Ext4 文件系统实例。

5.4. 缓存设计

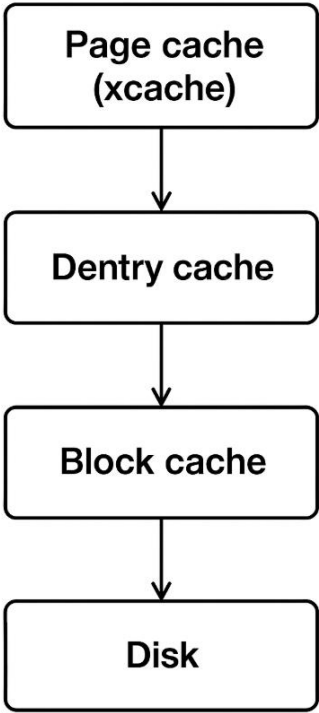


图 5.3 缓存层次图

为了弥合高速 CPU 与慢速磁盘之间的性能鸿沟，文件系统严重依赖缓存。StarryX 的文件系统中有多层缓存协同工作：

1. 页缓存（PageCache）：这是最重要的缓存层，用于缓存文件内容。当进程读写文件时，实际上是与内存中的 xcache 进行数据交换。如果数据在 xcache 中（命中），则无需访问磁盘。如果不在（未命中），文件系统层才会从磁盘读取数据块，填充到 xcache 的一个页面中，然后再提供给进程。它以页（Page）为单位进行管理，与内存管理系统紧密集成。我们将 PageCache 解耦为单个组件，将在第七章详细介绍。
2. 目录项缓存（DentryCache）：用于缓存路径查找结果。它存储了 key(父目录 inode, 文件名)到 inode 的映射。当内核需要解析一个路径时，它首先查询 dentry 缓存。如果命中，就可以立即获得目标的 inode，避免了从磁盘上反复读取目录文件来查找 dentry 的开销。这对于频繁访问相同文件或目录的场景，性能提升巨大。

```

1. pub type ReferenceKey = (usize, String); // (父目录指针, 文件名)
2.
3. pub struct Reference<M> {
4.     parent: Option<DirEntry<M>>,
5.     name: String,
6. }
7. impl<M> Reference<M> {
8.     pub fn key(&self) -> ReferenceKey {
9.         let address = self
10.             .parent
11.             .as_ref()
12.             .map_or(0, |it| Arc::as_ptr(&it.0) as usize);
13.         (address, self.name.clone()) // 快速哈希键
14.     }
15. }
16. pub struct DirNode<M> {
17.     ops: Arc<dyn DirNodeOps<M>>,
18.     cache: Mutex<M, BTreeMap<String, DirEntry<M>>>, // Direntry Cache
19.     pub(crate) mountpoint: Mutex<M, Option<Arc<Mountpoint<M>>>>,
20. }

```

块缓存(BlockCache): 这是最底层的缓存, 用于缓存磁盘的物理块(Block)。它位于具体文件系统和块设备驱动之间。即使是文件系统的元数据(如 inode 表、位图)的读写, 也会经过 block cache。pagecache 通常构建在 block cache 之上, 或者在某些设计中, page cache 直接管理文件数据页, 而 block cache 专注于元数据块

```

1. pub struct SeekableDisk {
2.     write_buffer: Box<[u8]>,
3.     write_buffer_dirty: bool,
4. }
5.
6. impl SeekableDisk {
7.     pub fn flush(&mut self) -> DevResukt<()> {
8.         if self.write_buffer_dirty {
9.             self.dev.write_block(self.block_id, &self.write_buffer)?;
10.            self.write_buffer_dirty = false; // 延迟写入
11.        }
12.        Ok(())

```



```
13.     }
14. }
```

5.5. 抽象文件

Unix 的“一切皆文件”哲学是操作系统设计的黄金标准，宏内核中有大量的抽象文件，比如套接字、管道等，对于 C 语言来说，它们通过函数指针表扩展并实现各种抽象文件，但这种方式抽象层次低、管理困难；而对于 Rust 语言，我们利用 Rust 的 Trait 设计了 FileLike trait，实现了对文件的高级抽象，通过 FileLike trait 我们具体实现了包括普通文件、普通目录、套接字、文件系统通知、进程文件描述符等丰富的抽象文件。

FileLike Trait 通过一系列操作高度抽象了文件的各种行为：

```
1. pub trait FileLike: Send + Sync {
2.     fn read()..           // 文件读
3.     fn write()..          // 文件
4.     fn stat()..           // 文件属性
5.     fn into_any()..       // 变为任意对象
6.     fn poll()..           // 轮询
7.     fn set_nonblocking().. // 设置阻塞
8.     fn is_nonblocking()..  // 是否阻塞
9.     fn from_fd()..        // 从 fd 构造
10.    fn add_to_fd_table()..  // 加入 fd_table
11.    fn get_location()..     // 得到位置
12.    fn len()..             // 得到长度
13. }
```

每个抽象文件打开时可能会有权限字段，我们将相关字段 FileFlags 与 FileLike 对象实体进行封装形成 StarryX 的抽象文件对象 XFile, 它继承了 FileLike 的方法，同时可以在进行操作前进行权限检查：

```
1. pub struct XFile {
2.     pub file: Arc<dyn FileLike>,
3.     pub flags: FileFlags,
4. }
5. impl XFile {
6.     // 权限检查
7.     pub fn validate(
```

```

8.         &self,
9.         required: FileFlags,
10.        forbidden: FileFlags,
11.    ) -> LinuxResult<&Arc<dyn FileLike>> {
12.        if self.flags.contains(required) && !self.flags.intersects(forbidden) {
13.            Ok(&self.file)
14.        } else {
15.            Err(LinuxError::EBADF)
16.        }
17.    }
18. }
19.
20. // 方法继承
21. #[inherit_methods(from = "self.file")]
22. impl XFile {
23.     pub fn read(&self, buf: &mut [u8]) -> LinuxResult<usize> {
24.         self.validate(FileFlags::READ, FileFlags::PATH)?.read(buf)
25.     }
26.     pub fn write(&self, buf: &[u8]) -> LinuxResult<usize> {
27.         self.validate(FileFlags::WRITE, FileFlags::PATH)?.write(buf)
28.     }
29.     pub fn into_any(self: Arc<Self>) -> Arc<dyn Any + Send + Sync> {
30.         self.file.clone().into_any()
31.     }
32.     pub fn stat(&self) -> LinuxResult<Kstat>;
33.     pub fn poll(&self) -> LinuxResult<PollState>;
34.     pub fn set_nonblocking(&self, nonblocking: bool);
35.     pub fn is_nonblocking(&self) -> bool;
36.     pub fn get_location(&self) -> Option<Location<RawMutex>>;
37.     pub fn len(&self) -> LinuxResult<u64>;
38. }

```

对于进程的文件描述符，我们通过一个文件描述符表进行维护，同时该表属于进程命名空间，在 clone 等行为发生时可能发生复制。我们学习了与 Linux 类似的文件描述符表与 Close-on-exe 的设置：

```

1. // 文件描述符表
2. pub struct FdTable {
3.     inner: RwLock<FlattenObjects<Arc<XFile>, AX_FILE_LIMIT>>,
4.     flags: RwLock<Bitmap<AX_FILE_LIMIT>>,

```

5. }

通过文件描述符表将文件描述符与抽象文件 XFile 正式建立了联系，当执行具体文件相关系统调用时具体执行以下几个阶段：

1. 从文件描述符表取得抽象文件
2. 判断是否具有访问权限
3. 该抽象文件是否为要求的目标文件
4. 执行具体操作

以 `pread` 为例：

1. `from_fd` 执行了前三个操作
2. 得到具体文件实体后执行 `read_at`

```
1. pub fn sys_pread64(  
2.     fd: c_int,  
3.     buf: UserPtr<u8>,  
4.     len: usize,  
5.     offset: __kernel_off_t,  
6. ) -> LinuxResult<isize> {  
7.     File::from_fd(fd, FileFlags::read, FileFlags::PATH)?  
8.         .read_at(buf, offset as _)  
9.         .map(|read| read as isize)  
10. }
```

5.6. 伪文件系统

伪文件系统（pseudo filesystem）是操作系统内核中一种不直接对应真实存储设备的文件系统，和 `ext4`、`xf`s、`fat` 这类“真实文件系统”不同，伪文件系统里的数据并不是来自磁盘，而是内核自己生成或维护的。它们主要有以下的作用：

1. 抽象统一

让应用程序通过“文件”接口来访问各种内核功能，避免专门系统调用。

2. 内核与用户空间的桥梁

内核可以通过伪文件系统向用户空间导出信息，用户可以通过读写伪文件来配置内核。

3. 增强灵活性

支持诸如进程管理、设备管理、调试监控等高级功能，而不依赖具体的硬件存储。

StarryX 实现了较为完善的伪文件系统，包括/dev、/dev、/tmp 等，它们通过统一的 Virt_file 和 Virt_fs 抽象实现了 axfs-vfs 的接口（FileNode 等），成为名义上的“文件系统实体”。

对于伪文件系统的实现，我们首先实现了结构体 VirtFs，其将 inode 通过 Slab 进行管理，并实现了具体的 FilesystemOps 与磁盘文件系统实例相对应：

```
1. /// Virtual filesystem implementation
2. pub struct VirtFs {
3.     name: String,
4.     fs_type: u32,
5.     inodes: Mutex<Slab<>>,
6.     root: Mutex<Option<DirEntry<RawMutex>>>,
7. }
8.
9. impl FilesystemOps<RawMutex> for VirtFs {
10.     fn name(&self) -> &str {
11.         &self.name
12.     }
13.
14.     fn root_dir(&self) -> DirEntry<RawMutex> {
15.         self.root.lock().clone().unwrap()
16.     }
17.
18.     fn stat(&self) -> VfsResult<StatFs> {
19.         Ok(dummy_stat(self.fs_type))
20.     }
21. }
```

对于伪文件系统中的虚拟文件，StarryX 实现了统一的 VirtNode 抽象，并实现了 vfs 的 NodeOps 与磁盘文件相统一：

```
1. /// Virtual filesystem node
2. pub struct VirtNode {
3.     fs: Arc<VirtFs>,
4.     ino: u64,
```

```

5.     pub(crate) metadata: Mutex<Metadata>,
6. }
7. impl NodeOps<RawMutex> for VirtNode {
8.     fn inode()..
9.     fn metadata()..
10.    fn len()..
11.    fn update_metadata()..
12.    fn filesystem()..
13.    fn sync()..
14.    fn into_any()..
15. }

```

在 VirtNode 之上我们封装了不同伪文件系统文件的具体实现，比如 VirtFile 和 VirtDevice，它们与实际磁盘的 inode 相统一：

```

1. // /dev 文件
2. pub struct VirtDevice {
3.     node: VirtNode,
4.     ops: Arc<dyn VirtDeviceOps>,
5. }
6.
7. pub trait VirtDeviceOps: Send + Sync {
8.     fn read_at(&self, buf: &mut [u8], offset: u64) -> VfsResult<usize>;
9.     fn write_at(&self, buf: &[u8], offset: u64) -> VfsResult<usize>;
10.    fn ioctl(&self, op: usize, argp: UserPtr<c_void>) -> VfsResult<isize>;
11. }
12.
13. #[inherit_methods(from = "self.node")]
14. impl NodeOps<RawMutex> for VirtDevice {
15.     fn inode()...
16.     fn metadata()...
17.     fn update_metadata()..
18.     fn filesystem()...
19.     fn sync()...
20.     fn into_any()...
21.     fn len()...
22. }
23.
24. impl FileNodeOps<RawMutex> for VirtDevice {
25.     fn read_at() ...

```

```

26.     fn write_at()...
27.     fn append()...
28.     fn set_len()...
29.     fn set_symlink()...
30. }

```

```

1. // /proc 文件
2. pub struct VirtFile {
3.     node: VirtNode,
4.     ops: Arc<dyn VirtFileOps>,
5. }
6.
7. pub trait VirtFileOps: Send + Sync {
8.     fn read_at(&self, buf: &mut [u8], offset: u64) -> VfsResult<usize>;
9.     fn write_at(&self, data: &[u8], offset: u64) -> VfsResult<usize>;
10.    fn len(&self) -> VfsResult<u64>;
11. }
12.
13. #[inherit_methods(from = "self.node")]
14. impl NodeOps<RawMutex> for VirtFile {
15.     fn inode()...
16.     fn metadata()...
17.     fn update_metadata()..
18.     fn filesystem()...
19.     fn sync()...
20.     fn into_any()...
21.     fn len()...
22. }
23.
24. impl FileNodeOps<RawMutex> for VirtFile {
25.     fn read_at() ...
26.     fn write_at()...
27.     fn append()...
28.     fn set_len()...
29.     fn set_symlink()...
30. }

```

6. 网络

6.1. 整体架构

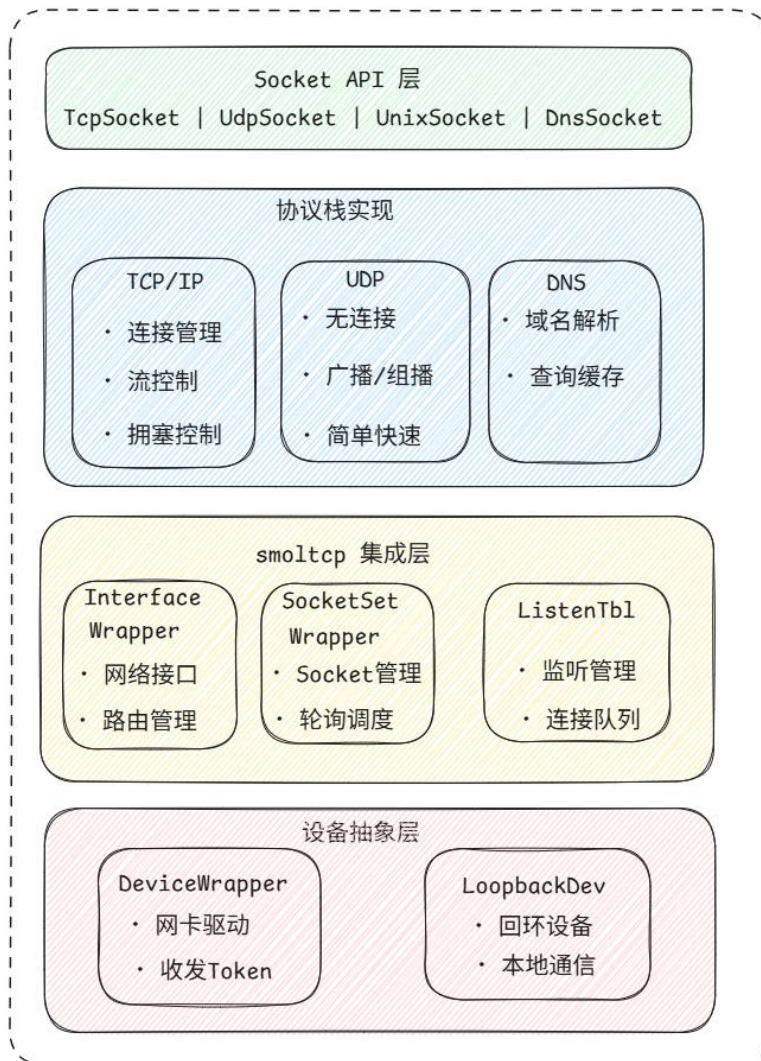


图 6.1 网络架构图

StarryX 的网络功能构建在一个三层架构之上，每一层都有明确的职责，实现了从硬件驱动到用户应用程序的完整数据通路。

1. **smoltcp (底层协议栈)**: 这是整个网络功能的核心引擎。**smoltcp** 是一个用 Rust 编写的、轻量级的、独立的 TCP/IP 协议栈。它的主要特点是无需标准库 (no-std) 和动态内存分配, 这使其非常适合在操作系统内核等受限环境中使用。**smoltcp** 负责处理网络协议中最复杂的部分, 包括: 解析和构建以太网帧、ARP、IPv4/IPv6、ICMP、UDP 和 TCP 数据包。管理 TCP 连接的状态机 (三次握手、四次挥手、拥塞控制等)。提供一个 **SocketSet** API, 用于管理一组活动的 TCP 和 UDP 套接字。

2. **axnet (ArceOS 网络抽象层)**: 这是连接底层 **smoltcp** 和操作系统其他部分的桥梁与适配器。它的职责是:

向下集成驱动: **smoltcp** 定义了一个通用的网络设备接口 (**Device trait**)。axnet

则负责将 axdriver 框架下的具体网卡驱动（如 ixgbe for Intel 10G NIC, virtio-net for QEMU）适配成 smoltcp 可以识别的设备。

向上提供接口: axnet 将 smoltcp 相对底层的 SocketSet API 封装成更符合内核使用习惯的接口。它管理着套接字的生命周期，并将网络事件（如收到数据包）与内核的同步机制（如 waitqueue）结合起来，以便让等待数据的任务可以被正确地阻塞和唤醒。

3. xcore/net (用户接口层): 这是网络栈的最高层，直接面向用户应用程序，负责实现标准的 API。

它将网络套接字（Socket）抽象成一个文件，实现了 FileLike trait。这使得用户进程可以像操作文件一样，使用 read(), write(), close() 等系统调用来收发网络数据。

它管理进程的文件描述符表与内核网络套接字之间的映射关系。当用户调用 socket() 时，xcore net 会请求 axnet 创建一个底层的 smoltcp 套接字，并将其包装成一个 XFile 对象，最后返回一个文件描述符给用户。

6.2. TCP 套接字

连接状态管理 TCP 连接的状态管理是协议实现的核心。axnet 使用原子变量 AtomicU8 来存储连接状态，确保状态变更的原子性。状态转换严格遵循 TCP 协议规范，每个状态转换都有相应的条件检查和错误处理。

```
1. pub struct TcpSocket {
2.     state: AtomicU8,           // 连接状态
3.     handle: UnsafeCell<Option<SocketHandle>>, // Socket 句柄
4.     local_addr: UnsafeCell<IpEndpoint>, // 本地地址
5.     peer_addr: UnsafeCell<IpEndpoint>, // 对端地址
6.     nonblock: AtomicBool,      // 非阻塞标志
7.     reuse_addr: AtomicBool,    // 地址重用标志
8. }
```

数据传输机制：TCP 的数据传输实现了完整的可靠传输机制。发送数据时，系统会将数据分割成适当大小的段，添加 TCP 头部，并通过 IP 层发送。接收数据时，系统会进行序号检查、重复检测和乱序重组。

流量控制实现：流量控制通过滑动窗口机制实现。接收方会在 TCP 头部的窗口字段中通告自己的接收窗口大小，发送方根据这个信息调整发送速度。axnet

的实现支持动态窗口调整，能够根据网络状况和接收方处理能力自动优化传输性能。

拥塞控制算法： axnet 实现了标准的 TCP 拥塞控制算法，包括：

- 1. **慢启动：** 连接建立初期，拥塞窗口从小开始，指数增长
- 2. **拥塞避免：** 当拥塞窗口达到阈值后，线性增长
- 3. **快速重传：** 检测到重复 ACK 时，立即重传丢失的段
- 4. **快速恢复：** 在快速重传后，快速恢复拥塞窗口大小

6.3. UDP 套接字

UDP 协议的实现相对简单，但在功能完整性和性能优化方面仍有许多考虑。

无连接特性： UDP 是无连接协议，不需要建立连接即可发送数据。axnet 的 UDP 实现充分利用了这一特性，提供了高效的数据报传输功能。每个 UDP Socket 可以同时与多个远程端点通信，无需为每个通信对象建立单独的连接。

广播和组播支持： axnet 的 UDP 实现支持 IP 广播和组播功能。广播功能允许向本地网络的所有主机发送数据，组播功能允许向特定的主机组发送数据。这些功能在网络发现、服务通告等场景中非常有用。

性能优化： 由于 UDP 协议的简单性，axnet 在 UDP 实现中进行了多项性能优化：

- 5. **零拷贝传输：** 在可能的情况下，避免数据拷贝操作
- 6. **批量处理：** 支持批量发送和接收操作，减少系统调用开销
- 7. **缓冲区管理：** 优化缓冲区分配和回收策略

6.4. 系统调用实现

StarryX 实现了完整的 POSIX 兼容网络系统调用：

Table：网络系统调用表

系统调用	功能	支持的套接字类型
sys_socket	创建套接字	TCP, UDP

sys_bind	绑定地址	TCP, UDP
sys_connect	连接远程地址	TCP, UDP
sys_listen	监听连接	TCP
sys_accept	接受连接	TCP
sys_send	发送数据	TCP, UDP
sys_recv	接收数据	TCP, UDP
sys_sendto	发送到指定地址	UDP
sys_recvfrom	从指定地址接收	TCP, UDP
sys_shutdown	关闭连接	TCP, UDP
sys_getsockname	获取本地地址	TCP, UDP
sys_getpeername	获取对端地址	TCP, UDP
sys_socketpair	创建套接字对	Unix

7. 组件化与抽象解耦

7.1. 设计理念

arceos 的设计强调组件化与模块化，StarryX 在设计时也希望遵从这一设计理念，将部分宏内核功能抽象为可供复用的独立组件，同时避免引入模块依赖，降低模块耦合度，提升系统的可维护性和可扩展性。

我们主要引入了以下宏内核相关组件：用户地址访问 `xuspace`，内存映射管理 `xvma`，页缓存模块 `xcache`，信号系统 `xsignal`，进程管理 `xprocess`，宏内核工具组件 `xutils`、宏内核应用测试套件 `xtest`。

7.2. Xuspace

`xuspace` 组件是 StarryX 中负责用户地址空间访问的核心模块，它为内核提供了安全、统一的用户空间内存访问接口，其封装了用户态地址访问的复杂性，确保内核在访问用户空间数据时的安全性和正确性。

在设计之初，xuspace 与 xcore 和 xapi 紧密耦合，由于初期时内存相关机制尚未实现，解耦于 StarryX 的组件（比如 xsignal）可以将用户指针转化为裸指针进行访问，但是这样的访问存在许多问题：

- 无法判断其合法性，无法安全访问用户地址
- 引入了大量对裸指针的 unsafe 操作
- 实现 cow 后可能导致在内核态发生缺页异常而发生致命错误

当实现内存延迟分配机制后，解耦于 xcore 的组件无法再正常访问用户地址空间，因此我们将 xuspace 从 xcore 中解耦成为一个独立组件，并提供抽象接口使其可以被其他系统所复用。

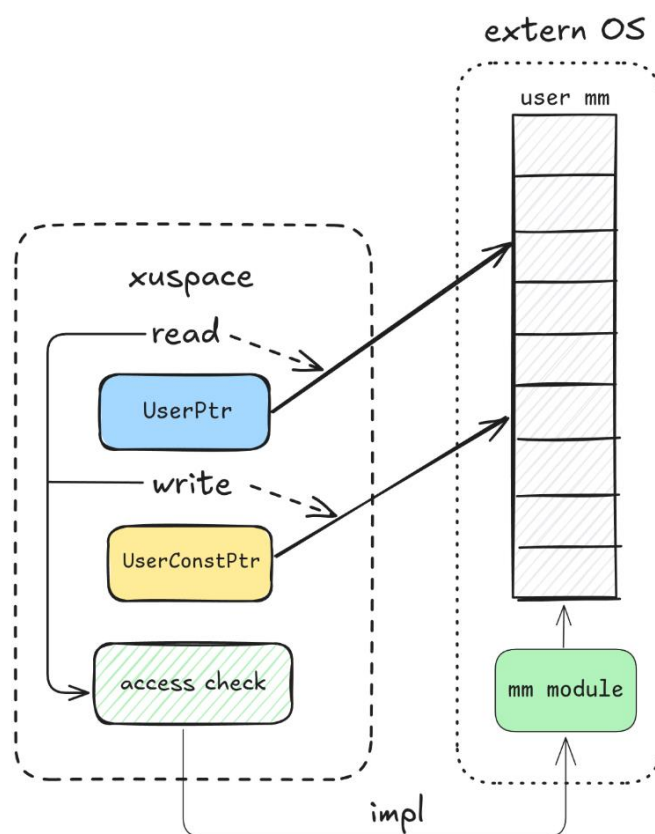


图 7.1 xuspace 架构图

在这个组件中，我们设计了两种指针类型 UserPtr 以及 UserConstPtr 分别对应用户传入的普通指针和 const 指针：

```
1. // basic ptr
2. pub struct UserPtr<T>(*mut T);
3. // const ptr
4. pub struct UserConstPtr<T>(*const T);
```

针对其具体功能我们设计了 `readabletrait` 抽象两种指针的行为，再为普通指针实现 `writabletrait`

另外需要实现对于用户空间安全访问的接口，这里接口需要实现：

- 维护用户空间与内核空间的严格边界，防止非法访问
- 提供安全的用户数据读取与写入接口
- 分配内存页避免发生页错误

这里的接口实现依赖于宏内核具体的虚拟地址空间管理方法，因此我们抽象了 `UserSpaceAccessTrait`，其暴露两个接口让内核实现用户内存访问的合法性检查，`UserSpaceAccess` 的剩下接口提供默认的内存访问方法给实现了该 trait 的 `uspace`，这些方法封装了 `UserPtr` 操作并调用内核实现的接口实现安全检查从而避免了直接操作 `UserPtr`。

```
1. pub trait Readable<T> {
2.     fn get_as_ref<A: UserSpaceAccess>(self, uspace: &A)
3.     fn get_as_slice<A: UserSpaceAccess>(self, uspace: &A, len: usize)
4.     fn get_as_null_terminated<A: UserSpaceAccess>(self, uspace: &A)
5. }
6.
7. pub trait Writeable<T> {
8.     pub fn get_as_mut<A: UserSpaceAccess>(self, uspace: &A)
9.     pub fn get_as_mut_slice<A: UserSpaceAccess>(self, uspace: &A, len:
10.         usize)
11.     pub fn get_as_mut_null_terminated<A: UserSpaceAccess>(self,
12.         uspace: &A)
13. }
```

其他 OS 只要实现 `UserAccessTrait` 中的两个用户内存地址检查接口就可以复用相关 API 实现用户地址空间访问。

7.3. Xprocess

`xprocess` 是实现 `StarryX` 进程管理的核心组件，它提供了完整的进程生命周期管理、进程间关系维护以及线程组织功能。它实现了数据管理与生命周期的管理的分离，将 `POSIX` 标准下的进程间关系由组件进行维护，而进程和线程的相关内部数据提供接口交由具体的 OS 实现，具有良好的灵活性和可扩展性。

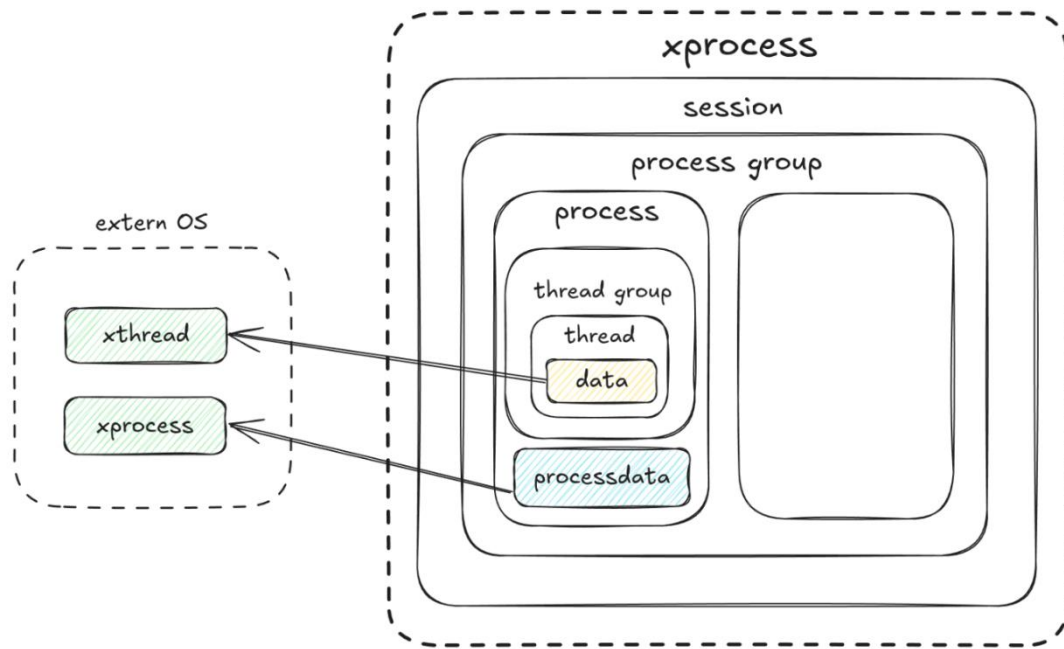


图 7.2 xprocess 框架图

其内部具体实现了基本的进程管理组织：

```

1. /// 线程
2. pub struct Thread {
3.     tid: ...      // 线程 id
4.     process: ...  // 所属进程
5.     data: ...     // 线程数据
6. }
7.
8. /// 线程组
9. pub struct ThreadGroup {
10.     threads: ...  // 线程集合
11.     exit_code: ... // 退出码
12.     group_exited: ... // 是否退出
13. }
14.
15. /// 进程
16. pub struct Process {
17.     pid: ...      // 进程 id
18.     is_zombie: ... // 僵尸进程
19.     tg: ...       // 线程组
20.     data: ...     // 进程数据接口
21.     children: ... // 子进程

```

```

22.     parent: ...      // 父进程
23.     group: ...       // 进程组
24. }
25.
26. /// 进程组
27. pub struct ProcessGroup {
28.     pgid: ...         // 进程组 id
29.     session: ...      // 会话
30.     processes: ...    // 进程集合
31. }
32.
33. /// 会话
34. pub struct Session {
35.     sid: ...           // 会话 id
36.     process_groups: ... // 所属进程组
37. }

```

这些数据结构实现了基本的进程层次管理与基本功能，在这基础上我们提供了进程创建退出等 api 让 OS 灵活地进行生命周期的维护：

```

1. /// 进程构建器
2. pub struct ProcessBuilder {
3.     data<T>(data: T) -> Self      // 设置进程数据
4.     build() -> Arc<Process>      // 构建进程实例
5. }
6.
7. /// 进程创建
8. impl Process {
9.     new_init(pid: Pid) -> ProcessBuilder // 创建 init 进程
10.    fork(pid: Pid) -> ProcessBuilder      // 创建子进程
11. }
12.
13. /// 进程状态控制
14. impl Process {
15.     exit(self: &Arc<Self>)           // 进程退出，转为僵尸状态
16.     group_exit(&self)                 // 标记整个线程组退出
17.     free(&self)                       // 释放僵尸进程资源
18. }

```

19.

20. /// 线程构建器等...

经过对进程层次管理和生命周期管理的实现，xprocess 为宏内核提供了稳定可靠的进程抽象和管理能力，同时保持了良好的可扩展性和可维护性。

7.4. Xcache

xcache 组件是 StarryX 中实现页面缓存管理的核心模块，基于 LRU 算法提供高效的文件页面缓存服务。该组件通过缓存文件数据页面，减少磁盘 I/O 操作，显著提升文件系统的访问性能（iozone 测例分数上升近 50%），同时提供完整的脏页管理和数据同步机制。在组件化 OS 中实现页缓存需要面对以下挑战

- 涉及多个模块，包括文件系统、内存管理和进程管理
- 原本独立的模块引入依赖，耦合度高（如文件系统引入内存管理依赖）
- 上层无法对缓存直接控制，回收困难

为了降低模块耦合度，保证底层 arceos 的 axfs 和 axmm 的独立性，我们将 PageCache 的实现抽象为一个独立组件，解除了多个模块的依赖关系，同时其不依赖特定的文件系统和内存管理系统实现，可以为不同类型的文件系统提供统一的缓存服务。

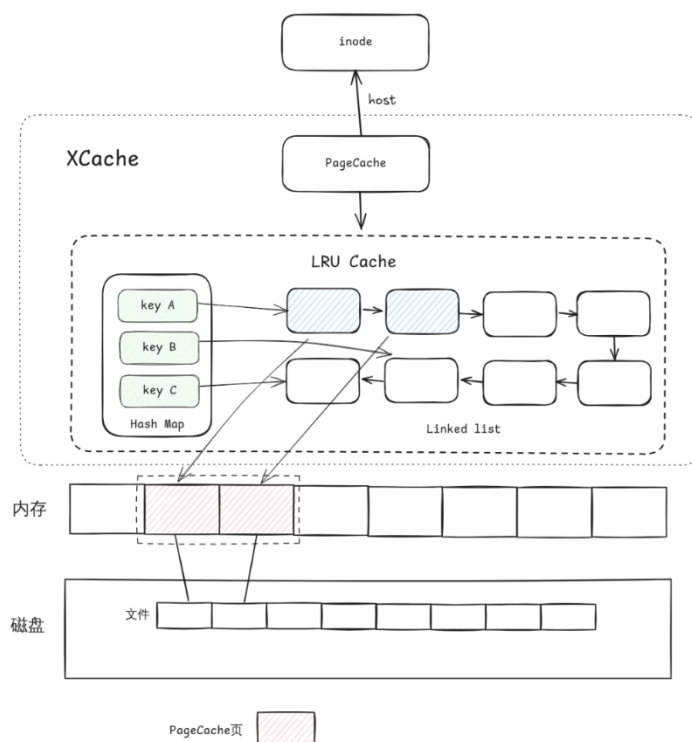


图 7.3 xcache 框架图

xcache 维护了一个 PageCache 数据结构，其与文件系统的 inode 相对应（类似于 Linux inode 与 addr_space 设计），其内部的 LRUCache 通过 HashMap 实现快速查找缓存页，同时基于 LRU 算法管理缓存页；PageCache 通过 trait 实现泛型抽象设计，将页缓存底层操作抽象为文件系统相关的读写操作和内存管理相关的页面分配及操作：

```
1. // 页缓存结构
2. pub struct PageCache<N: InodeOps, P: PageOps> {
3.     pub host: N, // 文件操作接口
4.     pages: Mutex<LruCache<u64, CachePage>>, // 缓存页集合（key 为偏移量）
5.     file_size: AtomicU64, // 文件大小
6.     _marker: core::marker::PhantomData<P>, // 页面操作接口
7. }
8.
9. // 缓存页
10. pub struct CachePage {
11.     pub addr: PhysAddr, // 页物理地址
12.     pub state: PageState, // 页面状态
13. }
```

与文件系统相关的操作为缓存未命中时对底层磁盘的读写操作：

```
1. /// 文件节点操作接口
2. pub trait InodeOps {
3.     fn read_at(&self, buf: &mut [u8], offset: u64) -> LinuxResult<usize>;
4.     fn write_at(&self, buf: &[u8], offset: u64) -> LinuxResult<usize>;
5. }
```

与内存管理相关的操作为缓存未命中时的物理页分配操作和命中时的读写操作：

```
1. /// 页面操作接口
2. pub trait PageOps {
3.     fn alloc_page() -> Option<PhysAddr>;
4.     fn dealloc_page(addr: PhysAddr);
5.     fn read_page(addr: VirtAddr, buf: &mut [u8]) -> LinuxResult;
6.     fn write_page(addr: VirtAddr, buf: &[u8]) -> LinuxResult;
7. }
```

在实现上述接口后，OS 即可实现自己的 PageCache 管理器。

PageCache 内部实现了完整的页面加载、写入操作、同步会写流程 (UpToDate->Dirty->WriteBack->ToWrite), 保证了页面缓存的数据数据一致性、操作安全性和性能优化:

```
1. pub enum PageState {
2.     UpToDate,           // 干净页面
3.     Dirty,              // 脏页
4.     WriteBack,          // 待回写 (提供一致性快照, 确保同步操作的原子性)
5.     ToWrite,            // 写回中 (提供 I/O 并发控制, 防止重复回写)
6. }
```

PageCache 也提供了灵活的页面回收机制, 包括局部回收, 全局回收以及 LRU 回收, 可以灵活应用于各个场景

```
1. impl PageCache {
2.     pub fn evict() ...      // 全局回收
3.     pub fn evict_range() ... // 回收指定范围
4.     pub fn evict_from_pos() ... // 从指定位置回收(ftruncate)
5.     pub fn evict_lru() ...   // 使用 LRU 算法回收
6. }
```

通过 xcache 组件, StarryX 实现了高效且可靠的页面缓存系统, 为文件系统提供了重要的性能优化基础设施, 同时保持了良好的模块化设计和扩展性。

xcache 目前仅实现了 Buffered IO, 没有实现对于进程管理相关的抽象 (mmap 通过 map_shared 分配), 这是我们未来进一步的改进和开发方向。

7.5. Xvma

xvma 组件是 StarryX 中专门处理文件支持的虚拟内存区域(mmap 分配)管理模块, 它实现了高效的按需加载内存映射机制。该组件专注于文件映射场景, 提供精确的地址范围管理和智能的页面加载策略。

在原本 arceos 的内存管理设计中 axmm 模块已经实现了地址空间 memory_set 的管理, POSIX 标准下 mmap 映射文件会引入虚拟内存区域与文件相关联的操作, 这与页缓存一样会使基座 OS 独立的模块引入依赖, 在与 arceos 的开发者交流后, 我们选择将这一层功能放在宏内核实现, 避免引入依赖, 保持模块的低耦合。

在组件内部我们实现了文件映射区域的有效管理，并且未来可以扩展到对所有 `mmap` 区域进行管理，与 `PageCache` 等组件配合工作完整实现 `mmap` 的所有功能：

```
1. /// 文件支持的内存映射区域
2. pub struct MmapRegion<F: VmFile> {
3.     pub range: VirtAddrRange,           // 虚拟地址范围
4.     pub file: F,                         // 支持的文件对象
5.     pub offset: isize,                   // 文件偏移量
6.     pub populated: Mutex<BTreeSet<VirtAddr>>, // 已加载页面集合
7.     pub align: PageSize,                 // 页面对齐大小
8. }
9.
10. /// 虚拟内存区域管理器
11. pub struct VmaManager<F: VmFile> {
12.     regions: Vec<MmapRegion<F>>,        // 内存映射区域集合
13. }
```

`mmap` 系统调用会对映射的文件页和内存区域在 `VmaManager` 注册虚拟内存区域，并对虚拟内存区域进行管理，完成精确的地址范围管理以及区域分割与合并，保持页面加载状态的一致性，并实现高效的地址查找和范围操作：

```
1. impl<F: VmFile> VmaManager<F> {
2.     add_region()...           // 添加映射区域
3.     find_region()...          // 查找包含地址的区域
4.     remove_overlapped()...     // 移除重叠区
5.     split_at_range()...       // 区域分割
6. }
```

通过 `xvmaStarryX` 实现了文件页的延迟加载策略，支持文件数据的按需读取和缓存，支持文件数据的按需读取和缓存；在发生缺页异常时，内核会先对发生缺页异常的地址进行快速区域查找，找到则读取文件数据，未找到则再交付给底层 `axmm` 执行缺页异常处理，实现了高效的文件页懒分配机制。

目前 `xvma` 主要支持了 `mmap` 的文件页映射管理，未来我们希望扩展 `xvma` 更多功能，使其可以成为一个高效独立管理 `mmap` 区域的组件，通过该组件减少进程管理、内存管理和文件系统间复杂的内核耦合关系。

7.6. Xsignal

xsignal 组件是 StarryX 中负责信号处理和管理的核心模块，它为内核提供了完整的 UNIX 风格信号处理机制。该模块实现了标准信号和实时信号的管理、信号动作配置、信号挂起队列管理以及跨平台的信号处理支持。

在传统的宏内核设计中，信号处理通常与进程管理、内存管理等子系统紧密耦合，这使得信号系统难以独立测试和维护。在 StarryX 的组件化设计中，我们将信号处理抽象为独立的 xsignal 组件，通过 trait 抽象解除与其他子系统的依赖关系，使其可以被不同的操作系统实现复用。在组件化 OS 中实现信号系统需要面对以下挑战：

- 多模块依赖：信号处理涉及进程管理、线程管理、用户空间访问等多个模块
- 平台相关性：不同架构下信号处理的细节存在差异

为了解决这些挑战，我们将 xsignal 设计为一个高度模块化的组件，通过 trait 抽象隐藏底层实现细节，同时保证信号处理的正确性和高效性。

在这个组件中，我们设计了核心的信号类型和管理结构：

```
1. // 信号类型
2. pub enum Signo { SIGHUP = 1, SIGINT = 2, ... SIGRT32 = 64 }
3. pub struct SignalSet(u64);
4. pub struct SignalInfo(pub siginfo_t);
5.
6. // 信号动作
7. pub struct SignalAction {
8.     pub flags: SignalActionFlags,
9.     pub mask: SignalSet,
10.    pub disposition: SignalDisposition,
11.    pub restorer: __sigrestore_t,
12. }
13.
14. // 挂起信号管理
15. pub struct PendingSignals {
16.    pub set: SignalSet,
17.    info_std: [Option<SignalInfo>; 32],    // 标准信号合并
18.    info_rt: [VecDeque<SignalInfo>; 33],    // 实时信号队列
19. }
```

针对其具体功能我们设计了 `WaitQueueTrait` 抽象线程等待行为，再通过复用 `xuserpace` 的 `UserSpaceAccessTrait` 实现安全的用户空间访问

```
1. // 线程等待抽象
2. pub trait WaitQueue: Default {
3.     fn wait_timeout(&self, timeout: Option<Duration>) -> bool;
4.     fn wait(&self);
5.     fn notify_one(&self) -> bool;
6.     fn notify_all(&self);
7. }
8.
9. // 信号处理函数中安全访问用户空间
10. fn handle_signal<A: UserSpaceAccess>(
11.     &self,
12.     uspace: &A,
13.     tf: &mut TrapFrame,
14.     // ...
15. ) -> Option<SignalOSAction>
```

另外需要实现对于信号处理的接口，这里接口需要实现：

- 维护进程级和线程级的信号状态管理
- 提供安全的信号投递与处理接口
- 支持标准信号合并和实时信号队列

这里的接口实现依赖于宏内核具体的进程管理和线程调度方法，因此我们抽象了双层管理架构，其暴露接口让内核实现信号处理的状态管理，其余接口提供默认的信号处理方法给实现了该 `trait` 的管理器，这些方法封装了信号操作并调用内核实现的接口实现状态同步从而避免了直接操作底层数据。

```
1. /// 进程级信号管理器
2. pub struct ProcessSignalManager<M, WQ> {
3.     pending: Mutex<M, PendingSignals>,
4.     pub actions: Arc<Mutex<M, SignalActions>>,
5.     pub(crate) wq: WQ,
6.     pub(crate) default_restorer: usize,
7. }
8.
9. /// 线程级信号管理器
```

```
10. pub struct ThreadSignalManager<M, WQ> {  
11.     proc: Arc<ProcessSignalManager<M, WQ>>,  
12.     pending: Mutex<M, PendingSignals>,  
13.     blocked: Mutex<M, SignalSet>,  
14.     stack: Mutex<M, SignalStack>,  
15. }
```

经过对双层信号管理和生命周期管理的实现，xsignal 为宏内核提供了稳定可靠的信号抽象和处理能力，同时保持了良好的可扩展性和可维护性。

其他 OS 只要实现 WaitQueue 抽象接口就可以复用相关 API 实现信号处理。

```
1. impl WaitQueue for MyWaitQueue {  
2.     fn wait_timeout(&self, timeout: Option<Duration>) -> bool { ... }  
3.     fn notify_one(&self) -> bool { ... }  
4.     // ... 其他方法  
5. }  
6.  
7. // 使用 xsignal 组件  
8. let proc_mgr = ProcessSignalManager::<MyMutex, MyWaitQueue>::new(...);  
9. let thread_mgr = ThreadSignalManager::new(Arc::new(proc_mgr));
```

通过 xsignal 组件，StarryX 实现了高效且可扩展的信号处理系统，为宏内核提供了完整的 UNIX 信号语义支持，同时保持了良好的模块化设计和跨平台兼容性。